IntelliTrade: Context-Enriched Multi-Agent Deep Reinforcement Learning for Algorithmic Trading

THTF9[1]

BSc Computer Science

Supervisor: Yun Fu

Submission date: 24th April 2026

[1]**Disclaimer:** This report is submitted as part requirement for the BSc Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged

Abstract

This report presents IntelliTrade, a context-enriched multi-agent deep reinforcement learning framework for algorithmic stock trading that combines large language model sentiment analysis signals with deep reinforcement learning trade execution and technical analysis. The central research question investigates whether grounding a Deep Q-Network trading agent in enhanced micro and macro economic context, extracted and structured by specialized multiple LLM agents querying a Causal-Temporal Knowledge Graph, improves risk-adjusted trading performance relative to baseline strategies.

The system exists of four subsystems: a data pipeline aggregating OHLCV price histories, earnings transcripts, and macroeconomic indicators for nine technology equities; a Neo4j-based Causal-Temporal Knowledge Graph encoding causal and temporal relationships extracted from financial text; a multi-agent LLM system (GPT-5.2 based) comprising four specialized agents that transform unstructured transcripts into structured signals and natural language narratives; and a Reinforcement Learning Deep Q-Network employing a dual-stream architecture with learned attention and gating mechanisms to dynamically fuse LLM-derived context with price-based technical indicators.

The agent is trained via 'experience replay' on data prior to January 2024 and evaluated on held-out data using strict walk-forward methodology from January 2024 to February 2026. The evaluation concludes concludes that the proposed hybrid medium to long term trading architecture delivers measurable improvements in cumulative return, Sharpe ratio, and maximum drawdown against, notably achieving approximately 10% alpha against the broader market (S&P500) for the same test period. Additionally, the evaluation demonstrated the positive impact of LLM generated signals to forecasting market prices, which in this case the augmentation improved the architecture's cumulative returns by 12%.

Contents

1 Introduction **2**
- 1.1 Problem Context and Motivation . . . 2
- 1.2 Research Goals and Personal Aims . . . 3
  - 1.2.1 Project Goals . . . 3
  - 1.2.2 Personal Aims . . . 3
- 1.3 Project Management . . . 4
- 1.4 Report Structure . . . 4

2 Context **5**
- 2.1 Large Language Models in Finance . . . 5
  - 2.1.1 From Domain-Specific Pre-Training to Supervised Fine-Tuning . . . 5
  - 2.1.2 The Shift Towards General Frontier Models with Context Engineering . . . 6
- 2.2 Deep Reinforcement Learning for Trading . . . 7
  - 2.2.1 Foundations of Reinforcement Learning . . . 7
  - 2.2.2 Deep Q-Networks (DQN) . . . 7
  - 2.2.3 Advantage Actor-Critic (A2C) . . . 7
  - 2.2.4 Deep RL Frameworks for Trading . . . 8
- 2.3 Knowledge Graphs for Financial Reasoning . . . 8
  - 2.3.1 Fundamentals of Knowledge Graphs . . . 8
  - 2.3.2 Temporal and Dynamic Knowledge Graphs . . . 9
  - 2.3.3 Causal-Temporal Knowledge Graphs for Financial Analysis . . . 9

3 Requirements and Analysis **11**
- 3.1 Problem Statement . . . 11
- 3.2 System Requirements . . . 12
  - 3.2.1 Scope and Constraints . . . 12
  - 3.2.2 Data Requirements . . . 12
  - 3.2.3 Functional Requirements . . . 12
  - 3.2.4 Non-Functional Requirements . . . 14
- 3.3 Requirements Analysis . . . 14
  - 3.3.1 Context Representation Trade-Offs . . . 14
  - 3.3.2 Multi-Agent vs. Monolithic LLM Architecture . . . 15
  - 3.3.3 Feature Fusion Strategy . . . 15
  - 3.3.4 Baseline Selection Rationale . . . 16

4 Design and Implementation **17**

4.1 System Architecture Overview . . . 17
4.2 Data Collection and Pre-Processing Pipeline . . . 18
4.2.1 Technical Indicators from Price Data . . . 18
4.2.2 Company Quarterly Fundamentals . . . 19
4.2.3 Macro-Economic Context . . . 19
4.3 Causal-Temporal Knowledge Graphs . . . 20
4.3.1 Graph Schema . . . 20
4.3.2 Vector Indexing and Semantic Search . . . 20
4.4 LLM Multi-Agent Signal Generation System . . . 21
4.4.1 FactExctractorAgent . . . 21
4.4.2 PeerAnalystAgent . . . 21
4.4.3 HistoricalAnalystAgent . . . 22
4.4.4 SynthesizerAgent . . . 22
4.4.5 Narrative Embedding . . . 22
4.5 Attention Mechanism and Context Gating . . . 22
4.5.1 Signal Attention . . . 23
4.5.2 Context Gating . . . 23
4.6 Markov Devision Process Formulation . . . 23
4.6.1 The Markov Property . . . 24
4.6.2 State Space . . . 24
4.6.3 Action Space and Transition Dynamics . . . 25
4.6.4 Reward Function . . . 25
4.7 Deep Q-Network Architecture . . . 25
4.7.1 Deep Q-Network Formulation . . . 25
4.7.2 Dual-Stream Network Architecture . . . 26
4.8 Training and Inference Strategy . . . 27
4.8.1 Training Procedure . . . 27
4.8.2 Inference and Deployment . . . 27

5 Backtesting and Results **29**
5.1 Evaluation Methodology . . . 29
5.1.1 Test Period and Market Context . . . 29
5.1.2 Performance Metrics . . . 30
5.2 Internal Ablation Results . . . 30
5.2.1 Aggregate Performance . . . 31
5.2.2 Risk-Adjusted Performance . . . 32
5.2.3 Variance Across Runs . . . 32
5.3 Comparison with Passive Baselines . . . 32
5.3.1 Buy-and-Hold of Individual Tickers . . . 32
5.3.2 Comparison with the Broad Market . . . 33
5.4 Comparison with External Work . . . 33
5.4.1 Comparison with Zhang et al. . . . 33
5.4.2 Comparison with TradingAgents . . . 33
5.4.3 Comparison with Public LLM Trading Leaderboards . . . 34
5.5 Qualitative Case Studies . . . 35
5.5.1 ASML: Strongest Gain from Context Enrichment . . . 35

5.5.2 TSM: A Useful Counterexample . . . 35
5.5.3 MSFT: Minimal Signal from Either Variant . . . 35
5.5.4 NVDA: Example Trade Trajectory . . . 35
5.6 Experimental Limitations . . . 36
5.7 Summary . . . 37

6 Conclusions and Evaluation **38**
6.1 Achievement of Project Goals . . . 38
6.1.1 Goal 1: Multi-Modal Financial Data Pipeline . . . 38
6.1.2 Goal 2: Causal-Temporal Knowledge Graph . . . 39
6.1.3 Goal 3: Multi-Agent LLM Signal Generation . . . 39
6.1.4 Goal 4: Context-Aware Deep Q-Network . . . 39
6.1.5 Goal 5: Empirical Evaluation Against Baselines . . . 39
6.2 Critical Evaluation of the Project . . . 40
6.3 Evaluation of Personal Aims . . . 41
6.4 Future Work . . . 41
6.5 Final Remarks . . . 42

Bibliography **43**

A NVIDIA Q2 Fiscal 2026 Earnings Call Transcript Excerpt **46**

B Sample from the Neo4j populated knowledge graph **58**

C Deployment Menu **59**
C.1 Environment Setup . . . 59
C.2 Train a Model . . . 60
C.3 Run Inference (Test) . . . 61
C.4 Compare Saved Models . . . 62
C.5 View Existing Analysis . . . 62

Chapter 1

Introduction

1.1 Problem Context and Motivation

The global algorithmic trading market was valued at USD 2.53 billion in 2025 and is projected to reach USD 4.33 billion by 2034 [1], which reflects the accelerating shift from discretionary, analyst based trading to systematic, algorithm based trading across financial markets. High-frequency trading firms, such as Citadel Securities, Jane Street, and Two Sigma, alone represent an estimated USD 11 billion market [2]. These firms are investing heavily in quantitative models and ultra-low-latency infrastructure; this growth has been accompanied by significant methodological evolution in the application of machine learning to financial forecasting and trading.

Early approaches of algorithmic trading relied on statistical time-series models such as ARIMA and GARCH for volatility forecasting [3]. The introduction of deep learning brought the notion of recurrent architectures to forecast prices: Long Short-Term Memory (LSTM) networks demonstrated improved performance on price prediction tasks by capturing long-range temporal dependencies in sequential data [4], whilst temporal convolutional networks (TCNs) offered parallelisable alternatives with dilated convolutions that efficiently model long sequences [5]. Attention mechanisms, popularised by Google's Transformer architecture in 2017 [11], enabled models to selectively focus on relevant historical time steps, which led to further gains in forecasting accuracy [6].

However, a fundamental short-sight persists across these supervised learning approaches: they frame trading as a prediction problem (forecasting future prices or returns) rather than a sequential decision-making problem (learning when and how to act). Deep reinforcement learning (DRL) addresses this disparity by allowing the model to learn adaptive trading policies directly from market interactions. Reinforcement learning based algorithms such as Deep Q-Networks (DQN) [13], Advantage Actor-Critic (A2C) [7], and Proximal Policy Optimization (PPO) [8] have been applied to equity trading with promising results, demonstrating the ability to learn non-trivial strategies that adapt to changing market regimes [14, 34].

Yet, a critical tension remains unresolved. All the algorithms and approaches mentioned so far only take into account quantitative signals (price, volume, technical indicators), but equities, evidently more so in today's markets, are also driven by qualitative signals earnings surprises, macroeconomic announcements, geopolitical events, and shifts in investor sentiment) that are difficult to encode in numerical feature vectors. This creates an underlying imprudence in predicting the movement

of markets: algorithms that can learn *how* to trade but lack an understanding of *why* the market is moving.

Recent advances in large language models (LLMs) have opened a new avenue for addressing this limitation. Models such as GPT-4 and domain-adapted variants like FinGPT [10], fine tuned on financial data, have demonstrated strong capabilities in financial comprehension, sentiment extraction, and event reasoning over unstructured documents (earnings call transcripts, news articles, regulatory filings). Integration of contextual understanding derived from LLM with reinforcement learning decision-making represents a frontier that is largely unexplored in systematic trading research. This project investigates whether grounding a DRL trading agent in domain specific, linguistically encoded context, extracted and structured by specialised LLM agents, can improve both sample efficiency and out-of-sample generalisation relative to algorithms that rely solely on price-based features.

1.2 Research Goals and Personal Aims

The overarching research question is: *Can grounding a DRL trading agent in semantically rich context, extracted and structured by specialized LLM agents querying a causal-temporal knowledge graph, improve risk-adjusted trading performance relative to baseline strategies that rely on either technical analysis alone or passive benchmarks?*

1.2.1 Project Goals

To address the research question, the project delivers five specific outputs:

- A data pipeline collecting and aggregating daily OHLCV price histories, quarterly earnings transcripts, fundamental metrics, and macroeconomic indicators for nine technology equities (2020–2026), with temporal alignment supporting walk-forward validation.
- A Causal-Temporal Knowledge Graph (Neo4j) encoding causal and temporal relationships extracted from earnings transcripts, enabling structured retrieval for LLM agents.
- A multi-agent LLM system (four specialised agents) that transforms unstructured earnings transcripts into structured signals and natural language narratives.
- A context-aware Deep Q-Network integrates LLM-derived signals for context alongside technical indicators.
- Empirical evaluation results comparing the context-enriched agent against baselines on out-of-sample data (January 2024 onward), demonstrating whether context enrichment improves risk-adjusted trading performance.

1.2.2 Personal Aims

This project serves several personal learning objectives beyond the technical contribution:

- Master transformer-based attention mechanisms in two contexts: understanding how LLMs process financial text and implementing attention for signal weighting in the trading agent.

- Develop practical expertise in reinforcement learning environment design, reward shaping, and applying deep RL algorithms to non-stationary environments with sparse, delayed rewards (skills directly transferable to post-training RL for large language models, such as RLHF).
- Gain hands-on experience with graph database and knowledge graph design and querying (Neo4j, Cypher) for temporal and causal knowledge representation.
- Learn to integrate multiple AI subsystems (LLMs, knowledge graphs, neural networks) into a cohesive end-to-end pipeline capable of operating on real-world financial data.
- Explore the emerging intersection of AI and quantitative finance, understanding both the opportunities and practical challenges of deploying machine learning in high-stakes decision-making environments.

1.3 Project Management

The project was executed using an iterative and sequential development approach. The first phase focused on data acquisition and pre-processing: raw market data and earnings transcripts were collected via the Alpha Vantage Premium API, cleaned, and aligned temporally. The second phase involved knowledge graph schema design and population, followed by implementation of the LLM agent pipeline with system prompt engineering and output validation. The third phase comprised RL agent architecture design, hyperparameter tuning, and training on historical data (2020-2024). The final phase consisted of out-of-sample evaluation, ablation studies, and performance attribution analysis to isolate the contribution of contextual signals.

1.4 Report Structure

The following section will cover the context of the project; exploring the landscape of algorithmic trading, particularly encompassing state of the art research in deep-learning with applications to finance and trading. After establishing the current environment, the concrete exigencies will be mapped out in the 'Requirements and Analysis' sections, motivated by ideas collected in the 'Context' section and initial experiments carried out in parallel. The 'Design and Implementation' section of the report outlines the progressive implementation of the system, from data collection and pre-processing, to the specific parameters and hyperparameters of the models (RL and LLM), and the train/test strategy. The system is then evaluated empirically using the results obtained from backtesting in the 'Backtesting and Results' section, where it is also compared against a number of baselines. Finally, the 'Conclusions and Evaluation' section reconciles the core project elements while addressing initial goals; it further serves to evaluate overall performance and propose potential improvements or extensions.

Chapter 2

Context

This chapter provides the technical background and current SOTA research underlying the three main components of the IntelliTrade system: large language models for financial text understanding, deep reinforcement learning for sequential trading decisions, and knowledge graphs for structured, temporal knowledge representation. Architectural decisions for IntelliTrade are motivated by deficiencies or lack of integrations as discussed in the proceeding sections.

2.1 Large Language Models in Finance

2.1.1 From Domain-Specific Pre-Training to Supervised Fine-Tuning

The application of natural language processing to financial text has progressed through three distinct generations. The first generation relied on lexicon-based methods in which hand-crafted dictionaries of positive and negative terms (e.g., the Loughran–McDonald word list [24]) were used to score the sentiment of earnings calls, analyst reports, and news articles. Although highly interpretable, these methods were unable to capture context-dependent meaning; the word "volatile", for instance, carries different connotations in a risk disclosure versus an earnings surprise.

The second generation introduced transfer learning via pre-trained language models. **FinBERT** [25], a BERT-based model further pre-trained on financial communication corpora, demonstrated that domain-adapted representations substantially improved sentiment classification accuracy on benchmarks such as the Financial PhraseBank and the FiQA opinion dataset. FinBERT outperformed both general-purpose BERT and traditional machine learning baselines, establishing that the specialised vocabulary and discourse conventions of financial text warranted dedicated pre-training. A concurrent variant [26], also called FinBERT, was pre-trained on a larger corpus of corporate filings, financial news, and analyst reports, achieving state-of-the-art results on financial sentiment analysis, question answering, and sentence boundary detection.

Finally, these ideas were scaled to decoder-only architectures.**BloombergGPT** [27], a 50-billion-parameter model trained on a 363-billion-token financial corpus ("FinPile") augmented with 345 billion tokens of general-purpose text, emphasizing the fact that mixed-domain pre-training could yield a model that significantly outperformed existing models on financial NLP tasks (including named entity recognition on SEC filings, financial sentiment analysis, and news classification) without sacrificing the general-purpose capabilities of frontier models. However, BloombergGPT

remains proprietary and inaccessible to the research community.

To democratise access, **FinGPT** [28] introduced an open-source framework that applies lightweight supervised fine-tuning (SFT) via low-rank adaptation (LoRA) [29] to general-purpose LLMs, adapting them to financial downstream tasks such as sentiment analysis, robo-advising, and algorithmic trading signal generation. FinGPT emphasised a *data-centric* approach, and demonstrated that capturing the general knowledge of frontier foundation models and applying parameter efficient fine tuning can be both cheaper and more powerful than pre-training. This paradigm (SFT on domain-specific instruction datasets) became the dominant strategy for building open-source financial LLMs through 2023 and 2024, spawning variants such as Fin-R1, DianJin-R1, Finance-LLaMA, and Finance-Qwen.

2.1.2 The Shift Towards General Frontier Models with Context Engineering

A more recent and significant shift in the field has been the growing evidence that general-purpose frontier models (when provided with well-engineered context rather than domain-specific fine-tuning) can match or exceed the performance of SFT-adapted financial models on a range of tasks. This follows from the transition from pre-training to supervised fine tuning by taking it a step further; relying completely on the weights of high reasoning frontier models while carefully engineering the context they consume.

The **FinChain** benchmark [30], which evaluates verifiable chain-of-thought financial reasoning across 26 LLMs, found that frontier proprietary models (GPT-5, Claude Sonnet 4.5, Gemini 2.5 Pro) consistently achieved the highest scores on multi-step symbolic financial reasoning. Finance-specific fine-tuned models (Fin-R1, Finance-LLaMA, Finance-Qwen) showed highly variable alignment quality, with some performing worse than general-purpose open-weight models of comparable size. Notably, the study observed that "structured supervision plays a critical role in achieving high step-level alignment", but that models fine-tuned narrowly for finance exhibited "more variable alignment quality" than those trained on broader mathematical corpora.

This finding is further corroborated by the recent **Alpha Arena** experiment [31], the first real-money evaluation of LLMs as autonomous trading agents. Alpha Arena allocated $10,000 of real capital to each of six frontier LLMs (GPT-5, Claude Sonnet 4.5, Gemini 2.5 Pro, Grok 4, DeepSeek Chat V3.1, Qwen 3 Max), with no task-specific fine-tuning, and tasked them with trading cryptocurrency perpetual futures using only numerical market data. The experiment revealed: (i) real behavioural differences in risk posture, directional bias, and trading frequency across models despite identical prompts; (ii) high sensitivity to small prompt perturbations and operational brittleness; and (iii) winning models succeeded through disciplined risk management rather than superior directional forecasting. Critically, none of these models were fine-tuned for trading.

These developments, primarily the exponential increase in reasoning capabilities of frontier models, suggest that for many financial reasoning tasks, the combination of a powerful general-purpose foundation model with carefully engineered retrieval and context provision may be preferable to domain-specific fine-tuning. Additionally, large context window foundation models are preferred for tasks requiring broad reasoning capabilities and the ability to integrate heterogeneous information sources. This observation directly informs the design choice in IntelliTrade to employ GPT-5.2 without fine-tuning, instead relying on structured prompting and adaptive retrieval from

the knowledge graph to provide task-relevant context.

2.2 Deep Reinforcement Learning for Trading

2.2.1 Foundations of Reinforcement Learning

Reinforcement learning (RL) formalises sequential decision-making as a Markov Decision Process (MDP) defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is a set of states, $\mathcal{A}$ a set of actions, $\mathcal{P}(s'|s,a)$ the transition dynamics, $\mathcal{R}(s,a)$ the reward function, and $\gamma \in [0,1)$ a discount factor. An agent interacts with the environment by observing state $s_t \in \mathcal{S}$, selecting action $a_t \in \mathcal{A}$ according to a policy $\pi(a_t|s_t)$, receiving reward $r_t = \mathcal{R}(s_t, a_t)$, and transitioning to state $s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$. The objective is to learn a policy π^* that maximises the expected discounted return $\mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$.

In the trading context, states encode the agent's current market observation (e.g., technical indicators, portfolio position, available capital), actions correspond to discrete trading decisions (buy, hold, sell), and the reward reflects the change in portfolio value or a risk-adjusted performance metric which the agent will optimize for. This Markov formulation naturally captures the sequential, path-dependent nature of trading, where today's decision constrains tomorrow's opportunity set.

2.2.2 Deep Q-Networks (DQN)

The **Deep Q-Network** (DQN) [13] marked a breakthrough in applying deep learning to reinforcement learning. DQN approximates the action-value function $Q(s,a;\theta)$ using a deep neural network parameterised by θ, trained to minimise the temporal-difference loss:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta)\right)^2\right]$$

where θ^- denotes the parameters of a periodically updated target network. Two key innovations stabilised training: (i) *experience replay*, which stores transitions (s_t, a_t, r_t, s_{t+1}) in a buffer and samples mini-batches uniformly to break temporal correlations; and (ii) the *target network*, a frozen copy of the Q-network updated at fixed intervals, which reduces the moving-target problem inherent in bootstrapped updates.

Mnih et al. demonstrated that DQN could achieve human-level performance across 49 Atari games using only raw pixel inputs [13]. In the trading domain, DQN has been widely adopted for its simplicity and interpretability: the Q-values for each action provide a direct ranking of the agent's confidence in buy, hold, and sell decisions, offering a degree of interpretability uncommon in policy-gradient methods. However, DQN is limited to *discrete* action spaces and can suffer from overestimation bias, where the max operator in the target computation systematically inflates Q-values. Extensions such as Double DQN [32] and Dueling DQN [33] address these limitations, though the fundamental discrete-action constraint remains.

2.2.3 Advantage Actor-Critic (A2C)

The **Advantage Actor-Critic** (A2C) algorithm maintains an actor $\pi(a|s;\theta_\pi)$ that selects actions and a critic $V(s;\theta_v)$ that estimates state value. The actor is updated via policy gradient using the

advantage $A_t = R_t - V(s_t; \theta_v)$ as a variance-reducing baseline:

$$\nabla_{\theta_\pi} J(\theta_\pi) = \mathbb{E}\left[\nabla_{\theta_\pi} \log \pi(a_t|s_t; \theta_\pi) \cdot A_t\right]$$

where $R_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n}; \theta_v)$ is the n-step return. A2C is the synchronous variant of the **Asynchronous Advantage Actor-Critic** (A3C) [7], which demonstrated that parallel actor-learners could stabilise training without experience replay, achieving state-of-the-art results on Atari whilst training on a single multi-core CPU rather than a GPU.

In trading, A2C offers several advantages over DQN. It naturally handles the exploration–exploitation trade-off through its stochastic policy, which can be regularised via an entropy bonus to prevent premature convergence. The advantage function reduces variance relative to vanilla policy gradients, leading to more stable learning in non-stationary financial environments. Comparative studies [34] have found that A2C tends to deliver superior *risk-adjusted* performance (higher Sharpe and Sortino ratios) even when DQN or PPO achieve higher raw cumulative returns, suggesting that A2C's policy structure favours conservative, steady strategies that minimise drawdowns.

2.2.4 Deep RL Frameworks for Trading

FinRL [35], the first open-source DRL framework for quantitative finance, implements a three-layer architecture (environment, agent, application) with a "training-testing-trading" pipeline that has become the standard evaluation methodology in the field. The framework provides implementations of multiple DRL algorithms (DQN, A2C, PPO, SAC) alongside environments compliant with the OpenAI Gym interface, enabling reproducible comparisons across strategies.

More recently, **TradingAgents** [36] extended this paradigm by introducing LLM-powered multi-agent collaboration with specialised analyst roles and structured debate mechanisms. In TradingAgents, multiple LLM agents with distinct personas (fundamental analyst, technical analyst, sentiment analyst, risk manager) deliberate via structured dialogue before reaching consensus trading decisions. The framework demonstrated that multi-agent debate could reduce individual model biases and improve risk-adjusted returns relative to single-agent baselines. However, the purely LLM-driven decision layer exhibited high operational costs (due to repeated inference calls during debate rounds) and sensitivity to prompt engineering.

IntelliTrade draws architectural inspiration from TradingAgents' multi-agent decomposition but replaces the purely LLM-driven decision layer with a DRL agent for trade execution, and engineers inter-agent knowledge flow through a structured temporal aware knowledge graph rather than free-form debate. This hybrid design aims to combine the contextual reasoning strengths of frontier LLMs for each quarter, with the sample-efficient, adaptive policy learning of deep RL for weekly re-balancing decision making, whilst reducing inference costs by caching structured signals in the graph.

2.3 Knowledge Graphs for Financial Reasoning

2.3.1 Fundamentals of Knowledge Graphs

A knowledge graph (KG) stores entities and relationships as triples (h, r, t), where h is the head entity, r is the relation, and t is the tail entity. Graph databases such as **Neo4j** implement the labelled property graph model with the Cypher query language, enabling expressive multi-hop

pattern matching that is critical for financial reasoning (e.g., "which companies supply NVIDIA, and how did their earnings calls discuss AI demand?"). Unlike relational databases, graph databases are optimised for traversing connections and do not require expensive join operations when querying multi-hop relationships.

2.3.2 Temporal and Dynamic Knowledge Graphs

Temporal Knowledge Graphs (TKGs) augment triples with a temporal dimension, yielding quadruples (h, r, t, τ), where τ denotes the timestamp or validity interval. This extension is particularly important for financial applications, where the meaning and relevance of relationships evolve over time (e.g., "Company A acquired Company B in Q2 2023"), and can capture time-dependent relationships and trends evident in quarterly earning reports.

FinDKG [38] demonstrated the practical value of LLM-generated dynamic knowledge graphs for finance. The framework uses a fine-tuned LLM (ICKG) to extract entities and relationships from financial news as event quadruples, then employs a KGTransformer architecture to perform link prediction and portfolio construction. FinDKG's event-driven KG outperformed existing thematic ETFs on both predictive accuracy and risk-adjusted returns, demonstrating that structured temporal knowledge can improve investment decision-making.

Complementary work by Shi et al. [39] showed that hybrid GCN-LSTM models operating over temporal knowledge graphs can capture temporal dependencies in stock price forecasting by jointly modelling the structural relationships between entities (via graph convolutions) and the sequential evolution of those relationships over time (via LSTM layers). These findings establish that temporal structure is not merely an auxiliary feature but a first-class component of effective financial reasoning systems.

2.3.3 Causal-Temporal Knowledge Graphs for Financial Analysis

The **Causal-Temporal Knowledge Graph** (CTKG) framework [40] extends standard TKGs by encoding not only *what* happened and *when*, but also *why*: causal edges link events to their downstream effects with temporal decay functions modelling diminishing relevance. When integrated with an agentic retrieval-augmented generation (RAG) architecture, the CTKG enables LLM agents to retrieve causally grounded context rather than semantically similar but causally irrelevant information, which is common in vector-based RAG systems. As vector-based RAG systems map context into high-dimensional embedding space, they are fundamentally limited to fuzzy similarity search and cannot guarantee retrieval of the most causally or temporally relevant information.

This approach aligns with recent work on causal discovery in financial markets. **CausalStock** [41], presented at NeurIPS 2024, demonstrated that lag-dependent temporal causal relations between stocks (discovered via deep end-to-end causal learning from news) significantly improve movement prediction across multiple markets. A time-aware causal graph is learned, where edges represent predictive relationships with specific time lags, enabling the model to forecast how news about one stock will propagate to peer related equities over subsequent trading days.

The combination of causal structure, temporal validity, and LLM-driven extraction positions CTKGs as a natural representation for the heterogeneous, evolving information landscape of financial markets. In IntelliTrade, the CTKG is implemented in Neo4j and populated by LLM-based

extraction pipelines processing earnings transcripts, corporate filings, and macroeconomic reports. Entities include companies, sectors, macroeconomic indicators, and financial events; relations include causal links (e.g., `[:EXPLAINED_BY]`), temporal succession (`[:NEXT_QUARTER]`), and structural connections (`[:IN_SECTOR]`). The multi-agent LLM system queries this graph via Cypher to retrieve causally and temporally relevant context for each trading decision, ensuring that the signals provided to the DRL agent are grounded in structured, and temporally accurate, domain knowledge rather than isolated text fragments that would have been retrieved in a vector based RAG setting.

Chapter 3

Requirements and Analysis

3.1 Problem Statement

The central challenge addressed by this project is the integration of heterogeneous information sources, both quantitative and qualitative, for algorithmic equity trading and market prediction. Traditional reinforcement learning approaches to algorithmic trading operate exclusively on structured numerical observations including price, volume, technical indicators derived from OHLCV data, and occasionally scalar fundamental metrics such as price-to-earnings ratios or earnings surprise magnitudes. Whilst these features capture market microstructure and short-term momentum, which can be usefult for high frequency trading, they are blind to the qualitative factors that drive much of the medium to long term price discovery process in equity markets.

Earnings call transcripts, for instance, contain dense and important information about management guidance, competitive positioning, supply-chain constraints, and demand outlook that is not readily encoded in historical price series. A company reporting strong revenue growth alongside deteriorating profit margins due to rising input costs presents a fundamentally different investment thesis than one achieving the same revenue growth through operational efficiency gains. Traditional technical indicators and ML algorithms cannot distinguish between these scenarios; they observe only the resulting price movement ex post. Similarly, macro-economic context (monetary policy stance, sectoral headwinds, geopolitical risk) shapes the interpretation of company-level fundamentals and broad industry price movement, yet this context is typically either ignored or naively proxied via market-wide indices.

As discussed previously, recent advances in large language models offer a potential solution: LLMs demonstrate strong capabilities in understanding financial text, extracting sentiment, and reasoning over complex narratives. However, directly deploying LLMs as trading agents introduces significant challenges. First, LLMs exhibit well-documented weaknesses in numerical reasoning and temporal consistency; asking an LLM to "decide whether to buy or sell" based on a transcript and price history often yields recommendations that are internally contradictory or fail to account for position constraints, not to mention extremely high variance decisions based on minor changes to the system prompt. Secondly, LLMs lack a disciplined learning mechanism; they do not improve their decision-making through repeated interaction with market feedback in the way that reinforcement learning agents do. Furthermore, unstructured retrieval systems that rely on embedding-based semantic search often retrieve contextually irrelevant information, for example

a query about "Q4 2024 guidance" may surface discussions from Q4 2023 simply because the language is similar, even though the market environment has fundamentally shifted.

This project addresses these challenges by proposing a hybrid architecture that combines the contextual reasoning strengths of LLMs with the adaptive, gradient-based learning of deep reinforcement learning, mediated through a structured knowledge graph representation of qualitative context.

3.2 System Requirements

3.2.1 Scope and Constraints

The system will operate on a defined universe of nine technology stocks (NVDA, ASML, TSM, AAPL, MSFT, AMZN, GOOGL, META, TSLA) over a six-year period (2020–2026). This scope is deliberately constrained to enable rigorous evaluation whilst remaining representative of high-growth, fundamentally-driven equity markets. The system shall implement a long-only trading strategy with weekly rebalancing, producing discrete decisions (buy, hold, sell) for each ticker independently.

The system shall operate in a simulated backtesting environment. All training shall occur on data prior to 5 January 2024; all evaluation shall occur on held-out data from from 5 January 2025 to 2 February 2026 using a strict walk-forward methodology that prevents look-ahead bias. The system shall not incorporate transaction costs or market impact models in the initial implementation, though the architecture shall be designed to accommodate these extensions.

3.2.2 Data Requirements

The system requires three classes of input data:

Quantitative Market Data: The system shall have access to daily OHLCV (Open, High, Low, Close, Volume) price histories for all nine tickers across the entire evaluation period. This data must be adjusted for corporate actions (splits, dividends) to ensure price continuity. The system shall compute technical indicators from this data to capture short- and medium-term price dynamics.

Qualitative Fundamental Data: The system shall have access to quarterly earnings call transcripts and corporate filings for all tickers. These documents provide the raw material from which contextual signals are extracted. Transcripts must be timestamped to their exact release date to enable strict temporal filtering during backtesting.

Macro-Economic Data: The system shall have access to quarterly aggregate indicators (CPI, inflation rate, unemployment rate, real GDP growth, Federal Funds Rate, 10-year Treasury yield) to contextualize company-level fundamentals within the prevailing economic regime.

3.2.3 Functional Requirements

FR1: Context Extraction and Structuring

Transforming unstructured earnings transcripts into structured representations that capture both the content of financial disclosures and the causal and temporal relationships between events. The representation shall support:

- Retrieval of facts based on semantic similarity (e.g., "find discussions of AI demand")
- Retrieval of facts based on temporal constraints (e.g., "find facts from Q3 2024 only")
- Retrieval of facts based on structural relationships (e.g., "find facts from peer companies in the same sector")
- Retrieval of causal explanations (e.g., "why did revenue increase?")

FR2: Multi-Perspective Analysis

Generating contextual trading signals from multiple analytical perspectives:

- Cross-sectional comparison: How does the company perform relative to sector peers?
- Temporal trend analysis: Is the company's financial trajectory improving or deteriorating over time?
- Narrative synthesis: What is the coherent investment thesis integrating all available information?

These signals shall be quantitative (numerical scores suitable for neural network consumption) and qualitative (natural language narratives providing interpretable context), primarily extracted by the LLM agents querying the knowledge graphs.

FR3: Context-Aware Decision-Making

Integrate LLM-derived contextual signals with price-based technical indicators to produce trading decisions. The integration mechanism shall satisfy the following properties:

- **Dynamic weighting:** The relative importance of different signals shall vary based on market conditions (e.g., fundamental signals may be more informative during or just after earnings season).
- **Reliability awareness:** The system shall adjust its reliance on contextual signals based on data quality indicators (e.g., reduce reliance when transcripts are unavailable or parsing fails).
- **Graceful degradation:** When contextual information is unavailable, the system shall revert to a purely technical strategy rather than failing or producing random decisions.

FR4: Policy Learning

Learn a trading policy through interaction with historical market data via deep reinforcement learning. The policy shall:

- Map the current state (market observations, contextual signals, portfolio position) to discrete actions (buy, hold, sell)
- Respect position constraints (long-only: cannot sell when flat, cannot buy when already long)
- Optimize for cumulative returns or risk-adjusted metrics over long horizons
- Learn from delayed rewards (today's decision affects future portfolio value)

FR5: Evaluation and Comparison

Support rigorous empirical evaluation against baseline strategies:

- Passive benchmark (buy-and-hold SPY)
- Technical-only variant (no LLM features)
- LLM-only variant (no gradient-based policy learning)

The evaluation framework shall compute performance metrics including cumulative return, Sharpe ratio, maximum drawdown, and win rate. The framework shall support ablation studies to isolate the contribution of individual components.

3.2.4 Non-Functional Requirements

NFR1: Temporal Integrity

The system shall strictly enforce temporal constraints during training and evaluation and prevent any lookahead bias. At any decision point t, the system shall have access only to information with timestamps strictly prior to t. This requirement is critical to prevent artificially inflating performance by allowing the agent to "see the future."

NFR2: Robustness to Missing Data

The system shall handle missing or incomplete data gracefully:

- If an earnings transcript is unavailable for a given quarter, as is sometimes the case using the Alpha Vantage API, the system shall continue operating using historical context and technical indicators
- If macro-economic data is delayed, the system shall use the most recent available data

NFR3: Computational Efficiency

The system shall complete training within reasonable time bounds (target: under 2 hours on a single GPU). Inference time for weekly rebalancing decisions shall be under 5 minutes per ticker to enable practical deployment in a live trading context.

3.3 Requirements Analysis

3.3.1 Context Representation Trade-Offs

The requirement for structured context retrieval (FR1) necessitates a choice between three architectural approaches:

Flat Vector Database: Store transcript embeddings in a vector store (e.g., Pinecone) and retrieve via cosine similarity. This approach is simple but fails to capture temporal and causal structure; a query for "Q4 2024 guidance" may retrieve Q4 2023 discussions simply because the language is similar.

Relational Database with Full-Text Search: Store transcripts in a relational schema with foreign keys representing temporal and structural relationships. This enables structured queries but struggles with semantic search (exact keyword matching vs. understanding "revenue growth" and "top-line expansion" as synonymous).

Graph Database with Vector Index: Store entities and relationships as a graph whilst maintaining vector embeddings on nodes for semantic search. This hybrid approach enables queries that combine graph traversal constraints (temporal, structural) with vector similarity ranking.

The analysis and thorough background research in the preceding chapter concludes that the graph-based approach is the only one that satisfies all four retrieval requirements in FR1. This motivates the design decision (detailed in Chapter 4) to implement the knowledge representation as a Neo4j graph with native vector indexing.

3.3.2 Multi-Agent vs. Monolithic LLM Architecture

The requirement for multi-perspective analysis (FR2) raises the question of whether to implement a single LLM prompt that produces all signals simultaneously, or decompose the task into specialized agents.

The monolithic approach is simpler but suffers from three limitations observed during preliminary prototyping:

1. **Inconsistent outputs:** A single prompt attempting to extract facts, compare peers, identify trends, and synthesize narratives frequently omits steps or produces contradictory statements.
2. **Limited context window utilization:** Asking the LLM to perform all tasks simultaneously compresses each sub-task into a fraction of the available context, whereas sequential agents can devote their full context window to each specialized task.
3. **Lack of modularity:** If the final narrative is implausible, it is difficult to diagnose whether the error originated in fact extraction, peer comparison, or trend detection.

The analysis concludes that a multi-agent pipeline with explicit intermediate outputs, and independent system prompts, better satisfies FR2 whilst enabling independent validation of each analytical step.

3.3.3 Feature Fusion Strategy

The requirement for context-aware decision-making (FR3) necessitates a fusion mechanism that integrates LLM-derived signals with technical indicators. Three candidate architectures were considered:

Early Fusion: Concatenate all features into a single vector and pass to a standard feedforward network. This is simple but treats all features uniformly; the high-dimensional LLM embeddings (384 dimensions) may dominate gradient updates, causing the network to underutilize technical indicators.

Late Fusion: Train separate networks on context and price features, then combine their outputs (e.g., via weighted averaging). This prevents feature dominance but requires manual tuning of the combination weights and does not enable learned interactions between modalities.

Learned Fusion with Gating: Process context and price features through separate pathways, then combine via a learned gating mechanism that adjusts the relative contribution based on data quality. This approach satisfies the dynamic weighting and graceful degradation requirements in FR3.

The analysis identifies learned fusion with gating as the only approach that satisfies all three sub-requirements of FR3, motivating the dual-stream architecture detailed in Chapter 4.

3.3.4 Baseline Selection Rationale

The requirement for comparative evaluation (FR5) necessitates careful selection of baseline strategies to isolate the contribution of each system component.

Internal Ablation Baseline:

- **Technical-Only DQN:** A variant of IntelliTrade trained without access to LLM features (context branch masked, agent operates solely on technical indicators and fundamental quality). Comparing full system vs. technical-only isolates the value added by contextual signals. If performance is similar, it suggests LLM context is redundant.

External Benchmarks:

- **Buy-and-Hold (SPY):** Passive benchmark representing market performance. Tests whether active trading outperforms passive indexing. If the system underperforms this baseline, it suggests that transaction costs or market efficiency render active strategies unviable.

- **TradingAgents Framework [36]:** An LLM-only multi-agent trading system where specialized agents deliberate via structured debate to reach consensus decisions. Comparing IntelliTrade's LLM-only variant against TradingAgents isolates architectural differences (knowledge graph vs. free-form debate, GPT-5.2 vs. their base model) whilst controlling for the core approach (multi-agent LLM reasoning).

- **Deep RL for Trading (Zhang et al.) [14]:** A technical-only DQN baseline trained on price-based features. Comparing IntelliTrade's technical-only variant against Zhang et al.'s implementation validates that the base DQN architecture and training procedure are sound before introducing LLM features.

Together, these comparisons enable a decomposition of system performance: the internal ablations isolate the contribution of LLM context and learned policy within IntelliTrade's architecture, whilst the external benchmarks establish whether IntelliTrade's innovations (context-enriched RL, knowledge graph retrieval) represent improvements over existing approaches in the literature.

Chapter 4

Design and Implementation

4.1 System Architecture Overview

The system presented in this project is a context-enriched, multi-agent reinforcement learning framework for algorithmic equity trading. Its central design challenge is the integration of heterogeneous information sources (structured market data and unstructured natural language) into a unified representation suitable for sequential decision-making in a simulated trading environment.

Figure 4.1 provides a high-level overview of the architecture, which is composed of four tightly coupled subsystems: (i) a data collection and pre-processing pipeline that aggregates OHLCV price histories, earnings transcripts, and macro-economic indicators; (ii) a Causal-Temporal Knowledge Graph (CTKG) implemented in Neo4j that structures unstructured financial text into queryable entities and relationships; (iii) a multi-agent LLM system (powered by GPT-5.2) that generates contextual signals via specialized analytical agents; and (iv) a Deep Q-Network that fuses LLM-derived context with technical indicators to produce trading decisions.

As illustrated in Figure 4.1, the architecture employs a dual-pathway design. The **context pathway** (left) processes qualitative information: earnings transcripts and filings are ingested into the CTKG, from which four specialized LLM agents extract structured signals and generate a natural language investment narrative; this narrative is embedded into a 384-dimensional vector via a sentence transformer. The **price pathway** (right) processes quantitative information: daily OHLCV data is transformed into seven technical indicators (RSI, momentum, volatility, moving average ratios, VWAP deviation, and Bollinger Band position) using NumPy-based computations. Both pathways converge at the DQN, which employs a dual-stream neural architecture with learned attention (to reweight LLM signals based on contextual relevance) and gating (to modulate the influence of the context branch based on data reliability). The network outputs Q-values for three discrete actions (buy, sell, hold), from which the agent selects the action with maximum expected return subject to position-dependent feasibility constraints.

A key architectural principle is the separation of concerns between these subsystems, which are connected via well-defined interfaces. This modularity allows each component to be developed, validated, and replaced independently whilst the overall pipeline retains coherence. The ticker universe comprises nine instruments: three semiconductor firms (NVDA, ASML, TSM), five large-cap technology companies (AAPL, MSFT, AMZN, GOOGL, META), and one electric vehicle

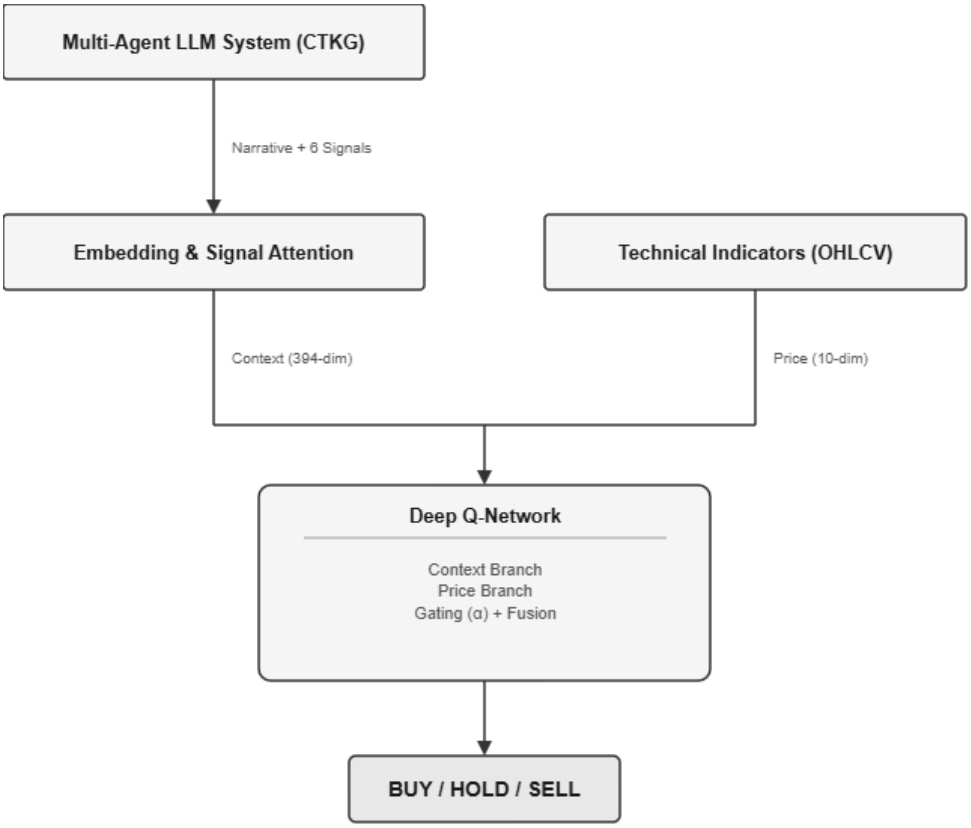


Figure 4.1: High-level system architecture

manufacturer (TSLA). This selection deliberately focuses on high-growth technology equities that exhibit shared macro-sensitivity and correlated price dynamics; such homogeneity permits a single RL model to be trained across the full instrument set and subsequently deployed on individual tickers, amortising training cost whilst preserving the ability to exercise the system's peer-comparison capabilities. Additionally, the technology focus allows for meaningful peer analysis extracted by the LLM agents from the knowledge graph at inference time.

4.2 Data Collection and Pre-Processing Pipeline

The data collection pipeline aggregates three classes of quantitative input and one class of qualitative input. All data is acquired prior to training via the Alpha Vantage Premium API, which provides unified access to market data, fundamental metrics, macro-economic indicators, and earnings transcripts. Data is pre-processed locally using (including cleaning, normalisation, and temporal alignment) before ingestion.

4.2.1 Technical Indicators from Price Data

Seven technical indicators are computed using Numpy from daily and weekly OHLCV (Open, High, Low, Close, Volume) price histories to capture short- and medium-term price dynamics. These indicators are selected to represent distinct aspects of market behaviour whilst minimising redundancy:

- **Relative Strength Index (RSI, 14-day)**: A momentum oscillator measuring the magnitude of recent price changes to identify overbought (RSI $>$ 70) or oversold (RSI $<$ 30) conditions

[20].

- **Price momentum (5-day)**: The percentage change in closing price over the previous five trading days, capturing short-term directional bias.
- **Historical volatility (20-day)**: The annualised standard deviation of daily log-returns over a rolling 20-day window, quantifying recent price dispersion.
- **Short-term moving average ratio (SMA 10/50)**: The ratio of the 10-day simple moving average to the 50-day simple moving average; values above 1.0 indicate short-term bullish momentum.
- **Medium-term moving average ratio (SMA 50/200)**: The ratio of the 50-day to 200-day simple moving average; values above 1.0 indicate medium-term bullish momentum (the "golden cross" regime).
- **Volume-weighted average price (VWAP) deviation**: The percentage deviation of the current closing price from the 20-day volume-weighted average price, normalised by volatility.
- **Bollinger Band position**: The relative position of the current price within the 20-day Bollinger Bands (standardised to $[0, 1]$, where 0 is the lower band and 1 is the upper band).

All indicators are normalised to $[-1, 1]$ or $[0, 1]$ ranges via min-max scaling computed over the training set to ensure numerical stability during gradient descent.

4.2.2 Company Quarterly Fundamentals

Quarterly company-level data comprises three components. First, company profile metadata (sector, industry, business description) is extracted to enable peer-group identification in the knowledge graph and cross-sectional benchmarking in the multi-agent signal generation pipeline. Second, earnings surprise records quantify the magnitude of deviation between reported quarterly earnings per share (EPS) and consensus analyst forecasts; these serve as discrete signals of market expectation error, where a positive earnings surprise (reported EPS exceeding consensus) indicates the company outperformed market expectations; a negative surprise indicates underperformance. These binary signals are subsequently encoded as features in the agent's state representation.

Third, the qualitative pipeline ingests quarterly earnings call transcripts and corporate filings (10-Q, 10-K). An example is the NVIDIA Q2 fiscal 2026 earnings call excerpt in Appendix A, which illustrates the density and volume of information that purely technical models neglect. These unstructured text documents are not directly consumable by the neural network policy; they require structured extraction and semantic compression, which is the responsibility of the LLM multi-agent system. Raw transcripts are parsed into atomic financial claims (facts) nodes in the knowledge graph via natural language processing, then stored for retrieval and aggregation by downstream analytical agents to generate contextual trading signals.

4.2.3 Macro-Economic Context

Six macro-economic indicators are aggregated at quarterly frequency: the Consumer Price Index (CPI), year-over-year inflation rate, unemployment rate, real GDP growth rate, the Federal Funds effective rate, and the 10-year Treasury yield.

Macro-economic context data has been included as a deliberate design decision motivated by the observation that sector-level fundamentals are frequently insufficient to explain equity price movements during periods of monetary policy transition (such as the 2022–2023 Federal Reserve tightening cycle) or systemic economic stress (such as the COVID-19 pandemic). By explicitly conditioning the LLM agents on the macro-economic state, the system can learn to modulate signal weights appropriately; for instance, downweighting peer-relative performance when all technology equities are simultaneously repriced due to rising interest rates.

4.3 Causal-Temporal Knowledge Graphs

The central knowledge representation component is a Causal-Temporal Knowledge Graph (CTKG) implemented in a Neo4j AuraDB instance, a cloud hosted native graph database which supports queying through the Cypher language. The decision to use a graph database (rather than a relational schema or a flat document store) is motivated by the need to explicitly encode two types of structural relationship that would otherwise remain implicit: causal relationships between explanatory factors and reported outcomes, and temporal relationships that agents can traverse between successive reporting periods. This design draws on the agentic RAG framework proposed by Chenhui Li [12], in which a structured graph serves as a persistent memory substrate for LLM agents, enabling multi-hop reasoning that would be infeasible over unstructured document collections.

4.3.1 Graph Schema

The graph schema defines seven node types: **Ticker** (an individual equity), **Quarter** (a discrete reporting period), **Fact** (an atomic, structured financial claim extracted from a transcript), **Reason** (a causal explanatory factor), **Transcript** (the source document), **EconomicContext** (the macro-economic snapshot for a given quarter), and **Summary** (an LLM-generated synthesis). Five relationship types encode the graph's semantic structure: `[:REPORTS_IN]` connects a Ticker to its associated Quarter nodes; `[:NEXT_QUARTER]` links consecutive Quarter nodes into a directed temporal chain; `[:HAS_FACT]` connects a Quarter to its extracted Fact nodes; `[:EXPLAINED_BY]` connects a Fact to a Reason, encoding a causal dependency; and `[:IN_SECTOR]` associates a Ticker with its industry sector for peer-group retrieval.

The graph is populated in batches to a single Neo4j AuraDB instance using data collected and preprocessed as described in Sections 4.2.2, and 4.2.3. Ticker and EconomicContext nodes are created first to establish the foundational structure; Quarter nodes are then instantiated and linked via `[:NEXT_QUARTER]` edges to form the temporal backbone. Fact nodes are ingested following transcript parsing by the `FactExtractorAgent` (Section 4.4). A sample screenshot of the knowledge graph for a single quarter is shown in Figure B.1, illustrating the resulting structured representation.

4.3.2 Vector Indexing and Semantic Search

Each Fact node stores a 384-dimensional dense vector embedding of its textual content in a property named `embedding`. A native Neo4j vector index is constructed over this property, enabling approximate nearest-neighbour (ANN) retrieval directly within graph queries. This avoids the need for a separate vector store (such as Pinecone or ChromaDB), keeping retrieval co-located with the graph traversal logic and reducing overall system complexity. Combining symbolic graph

traversal and sub-symbolic vector similarity constitutes the hybrid retrieval pattern upon which the multi-agent system depends; this is a design principle explicitly advocated in the context-enriched agentic RAG literature [12].

4.4 LLM Multi-Agent Signal Generation System

The signal generation layer is implemented as a pipeline of four specialised, context-enriched LLM agents, each assigned a distinct analytical role through purposely crafted system prompts. This multi-agent decomposition is preferred over a single monolithic prompt to enforce a separation of reasoning tasks that would otherwise compete for limited context window capacity, and produce modular intermediate outputs that can be independently validated and reused across multiple downstream consumers. Moreover, this addresses the variance of signal generation as a result of perturbations to the system prompt as described previously by coalescing signals from multiple highly focused agents. All agents are powered by GPT-5.2 (deployed via Azure AI Foundry), selected on the basis of its performance on domain-relevant evaluation benchmarks. On Finance Agent v1.1 [15] (a benchmark of 537 expert-authored questions covering search and retrieval tasks over SEC filings performed by financial analysts), GPT-5.2 achieves competitive performance whilst remaining substantially more cost-effective than frontier reasoning models such as Claude Opus 4.6. On DeepSearchQA [16] (a 900-prompt benchmark for multi-hop question-answering requiring systematic collation of fragmented information from disparate sources), GPT-5.2 achieves state-of-the-art performance with an F1 score competitive with Gemini Deep Research whilst operating at lower inference cost. Azure AI Foundry provides native support for custom system prompts (enabling role-specific agent behaviour) and structured JSON output modes via function calling (eliminating the need for error-prone post-hoc regex parsing and reducing fact extraction errors).

4.4.1 FactExctractorAgent

The `FactExtractorAgent` is the entry point of the pipeline. Prompted as a "precise financial fact extraction bot", its sole responsibility is to parse raw earnings transcripts into a structured JSON schema comprising four fields per extracted fact: metric name, reported value, period-over-period change, and a sentiment polarity label. This strict output format transforms unstructured prose into graph-ready data, subsequently ingested into Neo4j as Fact nodes. Constraining this agent to a single, well-defined task reduces hallucination risk and ensures that downstream agents receive consistently structured inputs when querying the knowledge graph.

4.4.2 PeerAnalystAgent

The `PeerAnalystAgent` performs cross-sectional benchmarking. It queries the CTKG vector index to retrieve semantically similar Fact nodes from companies classified in the same sector via the `[:IN_SECTOR]` relationship. Given these peer facts, the agent (prompted as a "comparative financial analyst") assesses the subject company's performance relative to its cohort and produces a scalar Peer Score in the range $[-1.0,\ 1.0]$, where positive values indicate outperformance. Relative scoring is important because absolute financial metrics are often less informative than their sector-relative counterparts, particularly for growth-oriented technology equities.

4.4.3 HistoricalAnalystAgent

The `HistoricalAnalystAgent` exploits the `[:NEXT_QUARTER]` temporal chain within the CTKG to traverse a company's own historical Fact nodes across multiple reporting periods. By combining this graph traversal with vector similarity search over the company's past embeddings, the agent identifies trends in key financial metrics. Its outputs (revenue trend, margin trend, and historical momentum scores) capture time-series dynamics that would otherwise require separate modelling components if the causal-temporal graph structure were absent.

4.4.4 SynthesizerAgent

The `SynthesizerAgent` acts as the terminal stage of the context generation pipeline. Prompted as a "senior investment strategist", it receives the outputs of all preceding agents alongside the macroeconomic context retrieved from the CTKG EconomicContext nodes, and produces two forms of output: a natural language narrative of approximately 300 words and a set of six structured scalar signals (revenue trend, margin trend, peer relative performance, historical momentum, risk level, and overall sentiment).

4.4.5 Narrative Embedding

The 300-word natural language narrative produced by the `SynthesizerAgent` must be transformed into a numerical representation consumable by the neural network policy. Raw text cannot be directly processed by the Q-network; the embedding step bridges the gap between symbolic linguistic content and the continuous vector space required for gradient-based learning by encoding it into a fixed 384-dimensional dense vector using the `all-MiniLM-L6-v2` sentence transformer model [17]. The open source `all-MiniLM-L6-v2` model from HuggingFace was selected for its balance of semantic quality (MTEB score: 56.0 [18]) and low inference latency suitable for local deployment.

This 384-dimensional embedding captures the holistic semantic content of the investment thesis (including sentiment polarity, identified risks, and comparative context) in a form that preserves geometric relationships; narratives with similar financial implications map to nearby points in the embedding space. The embedding vector is concatenated with the six structured scalar signals and passed to the attention and gating mechanism (Section 4.5) for consumption by the RL policy network.

4.5 Attention Mechanism and Context Gating

A critical design consideration in fusing LLM-derived context with price-based signals is the variable quality and reliability of the context. Transcripts may be absent, partially parsed, or internally contradictory across agents; treating the LLM output as uniformly trustworthy for all tickers and time-steps would introduce noise into the policy gradient signal during training. To address this, the architecture employs two learned weighting mechanisms: signal-level attention and context-level gating. Studies as well as internal benchmarking during experimentation for the use of multiplicative gates to modulate multimodal information flow has been shown to substantially improve fusion robustness in scenarios where input modalities exhibit variable reliability [19].

4.5.1 Signal Attention

The six structured signals from the `SynthesizerAgent` (revenue trend, margin trend, peer relative performance, historical momentum, risk level, and overall sentiment) exhibit non-stationary importance and shared latent dependencies. Revenue trend may dominate during an earnings-driven rally, whilst peer relative performance becomes uninformative during a market-wide sell-off; similarly, margin and revenue trends often co-vary in response to common shocks. Therefore, a uniformly weighted concatenation treats all signals equally and ignores these dependencies.

The system employs scaled dot-product attention [11] to dynamically reweight signals based on contextual relevance. Let $\mathbf{x}_t^{\text{raw}} \in \mathbb{R}^6$ denote the raw signal vector. The attended output is:

$$\mathbf{x}_t = \text{softmax}\left(\frac{(\mathbf{W}^Q \mathbf{x}_t^{\text{raw}})(\mathbf{W}^K \mathbf{x}_t^{\text{raw}})^\top}{\sqrt{d_k}}\right)(\mathbf{W}^V \mathbf{x}_t^{\text{raw}}) \tag{4.1}$$

where $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_k \times 6}$ and $\mathbf{W}^V \in \mathbb{R}^{6 \times 6}$ are learned projection matrices trained end-to-end with the Q-network. The attended signal vector $\mathbf{x}_t$ is then passed to the Context Branch.

4.5.2 Context Gating

At the branch-fusion level, a learned scalar gate $\alpha \in [0, 1]$ modulates the contribution of the Context Branch (which processes LLM-derived signals) relative to the Price Branch (which processes technical indicators). The gate is conditioned on context reliability metadata:

$$\alpha = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{m}_t + \mathbf{b}_1) + b_2) \tag{4.2}$$

where $\mathbf{m}_t \in \mathbb{R}^5$ encodes context reliability (fact count, agent agreement, parse success, transcript recency, and macro-data availability), $\mathbf{W}_1 \in \mathbb{R}^{16 \times 5}$ projects to a 16-dimensional hidden layer, and $\mathbf{W}_2 \in \mathbb{R}^{1 \times 16}$ projects to a scalar logit. The fused representation is:

$$\mathbf{z}_{\text{final}} = \mathbf{z}_{\text{price}} + \alpha \cdot \mathbf{z}_{\text{context}} \tag{4.3}$$

When context reliability is low (e.g., transcript unavailable, no facts extracted), the gate closes ($\alpha \approx 0$) and the agent reduces to a purely technical strategy. When rich, internally consistent context is available, the gate opens ($\alpha \approx 1$), fully incorporating the LLM signal. This graceful degradation distinguishes the architecture from concatenation-based fusion approaches that assume constant context quality, and the agent can learn when and to what extent it should 'attend' to LLM-generated signals to maximize its reward function.

4.6 Markov Devision Process Formulation

The trading decision problem is formalised as a Markov Decision Process (MDP). An MDP provides the mathematical foundation for sequential decision-making and is the standard framework underlying modern reinforcement learning algorithms. Formally, an MDP is defined by the tuple:

$$\mathcal{M} = \langle \mathcal{S},\ \mathcal{A},\ \mathcal{P},\ \mathcal{R},\ \gamma \rangle \tag{4.4}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability kernel, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is a discount factor that

determines the relative weight of immediate versus future rewards.

4.6.1 The Markov Property

A central requirement of the MDP framework is the Markov property, which states that the future evolution of the process depends only on the current state and not on the history of past states:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots) \;=\; P(s_{t+1} \mid s_t, a_t) \tag{4.5}$$

In the context of this system, the Markov property is approximately satisfied by constructing the state vector to be sufficiently informative: it encodes the current price-based technical indicators, the agent's current portfolio position, the LLM narrative embedding (capturing recent fundamental trends), and the macro-economic context. By concatenating these features into a single 485-dimensional state observation, the system ensures the agent does not require access to its history of past states to make a well-informed decision at each time step. This approach is consistent with that adopted by [14], in which sufficiently rich state representations are used to satisfy the Markov assumption in practice.

4.6.2 State Space

The state vector $\mathbf{s}_t \in \mathbb{R}^{408}$ presented to the policy network at each decision step t is a concatenation of six feature groups:

$$\mathbf{s}_t = [\, \mathbf{e}_t \parallel \mathbf{x}_t \parallel \mathbf{f}_t \parallel \mathbf{q}_t \parallel \mathbf{n}_t \parallel \mathbf{m}_t \parallel p_t \,] \tag{4.6}$$

where each component encodes a distinct aspect of the trading context:

$\mathbf{e}_t \in \mathbb{R}^{384}$ is the **narrative embedding**, a dense vector representation of the 300-word investment memorandum produced by the `SynthesizerAgent` (Section 4.4). This embedding captures the holistic semantic content of the LLM's qualitative assessment, including sentiment polarity, identified risks, and comparative context.

$\mathbf{x}_t \in \mathbb{R}^6$ is the vector of **attended structured signals**, comprising six scalar values output by the `SynthesizerAgent`: revenue trend, margin trend, peer relative performance, historical momentum, risk level, and overall sentiment. These signals have been weighted by the attention mechanism (Equation 4.1) to reflect their relative importance under current market conditions.

$\mathbf{f}_t \in \mathbb{R}^7$ is the vector of **technical indicators** (RSI, momentum, volatility, and moving average features) computed from raw OHLCV price data as outlined in Section 4.2.1. These values represent assertive price-derived signals that are independent of the LLM pipeline.

$\mathbf{q}_t \in \mathbb{R}^2$ is a one-hot encoding of **binary fundamental quality**, derived from the most recent earnings surprise record (Section 4.2.2). The encoding is $\mathbf{q}_t = [1, 0]$ for a negative earnings surprise (reported EPS below consensus) and $\mathbf{q}_t = [0, 1]$ for a positive surprise (reported EPS exceeding consensus).

$\mathbf{n}_t \in \mathbb{R}^3$ is a one-hot encoding of **narrative regime state** (bullish, neutral, or bearish), derived via zero-shot classification of the narrative embedding $\mathbf{e}_t$. The regime is determined by computing cosine similarity between $\mathbf{e}_t$ and three fixed reference embeddings representing archetypal investment narratives for each sentiment class; the class with highest similarity is selected. This categorical representation provides the network with an interpretable, coarse-grained signal of market sen-

timent that complements the continuous embedding $\mathbf{e}_t$. One-hot encodings have been manually checked against the raw narrative text to ensure they are being categorized correctly.

$\mathbf{m}_t \in \mathbb{R}^5$ is the vector of **context reliability metadata**, which consists of five scalar indicators of LLM pipeline quality: (i) the number of facts extracted from the earnings transcript, (ii) the agreement score between the `PeerAnalystAgent` and `HistoricalAnalystAgent` outputs, (iii) the transcript parse success rate, (iv) the age (in days) of the most recent transcript, and (v) a binary flag indicating whether macro-economic data is available for the current quarter. This metadata is consumed by the context gate (Equation 4.2) to modulate the influence of the LLM-derived signals.

$p_t \in \{0, 1\}$ encodes the **current portfolio position**: 0 indicates the agent is flat (holding no position), whilst 1 indicates the agent is fully long (holding a 100% allocation to the ticker). This feature enables the network to condition its action selection on whether it currently holds a position, which is necessary to enforce the long-only constraint (Equation 4.7).

4.6.3 Action Space and Transition Dynamics

The system implements a long-only constraint where no short positions are permitted, to maintain a simple reward policy for the agent to optimize for. This restricts the discrete action space to $\mathcal{A} = \{\text{BUY}, \text{HOLD}, \text{SELL}\}$ and enforces a position-dependent feasibility constraint. The feasible action set at time t is:

$$\mathcal{A}(\mathbf{s}_t) = \begin{cases} \{\text{BUY}, \text{HOLD}\} & \text{if } p_t = 0 \quad \text{(flat)} \\ \{\text{SELL}, \text{HOLD}\} & \text{if } p_t = 1 \quad \text{(long)} \end{cases} \tag{4.7}$$

This constrained formulation simplifies the policy optimisation problem and reflects realistic constraints applicable to many institutional trading contexts. Invalid actions are 'masked' prior to Q-value selection, eliminating the need to penalise them during training.

4.6.4 Reward Function

The reward signal is defined as the single-period logarithmic return, realised only while the agent holds a long position:

$$r_t = \ln\left(\frac{P_{t+1}}{P_t}\right) \cdot \mathbb{1}[a_t = \text{BUY} \ \text{ or } \ p_t = 1] \tag{4.8}$$

where P_t denotes the closing price at step t. Logarithmic returns are preferred over simple returns because they are approximately normally distributed and additive across time steps, which stabilises the gradient signal during training.

4.7 Deep Q-Network Architecture

4.7.1 Deep Q-Network Formulation

Given the MDP defined above, the goal of the RL agent is to learn a policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximises the expected discounted cumulative return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k} \tag{4.9}$$

Direct policy search over the 408-dimensional state space is intractable. Instead, the system employs the Deep Q-Network (DQN) algorithm [13], as applied to the trading domain by [14]. DQN approximates the optimal action-value function $Q^*(s, a)$ (the expected return of taking action a in state s and subsequently following the optimal policy) using a deep neural network with parameters θ:

$$Q^*(s, a) = \max_{\pi} \mathbb{E}_{\pi}[\, G_t \mid s_t = s,\ a_t = a \,] \tag{4.10}$$

The network is trained by minimising the temporal difference (TD) loss at each update step:

$$\mathcal{L}(\theta) = \mathbb{E}_{\,(s,a,r,s') \sim \mathcal{D}} \left[\Big(y_t \ - \ Q(s_t,\, a_t;\, \theta) \Big)^2 \right] \tag{4.11}$$

where the TD target y_t is computed using a periodically updated *target network* with frozen parameters θ^-:

$$y_t = r_t + \gamma \cdot \max_{a'} \ Q\big(s_{t+1},\, a';\, \theta^-\big) \tag{4.12}$$

Decoupling the regression target from the parameters currently being updated (via θ^-) substantially improves training stability. This technique, alongside experience replay described in Section 4.8, constitutes the two principal algorithmic innovations of DQN [13]. Ultimately, DQN allows for training the RL model over a high dimensional and complex state space by using a feed forward neural network to approximate the Q-function; which action to take at each state.

4.7.2 Dual-Stream Network Architecture

The Q-network for IntelliTrade employs a dual-stream architecture with 60,530 trainable parameters, processing price-based and context-based features through separate pathways before fusion.

Context Branch (59,800 parameters): Processes the narrative embedding, attended signals, and context metadata. Input dimensionality is 394 (comprising the 384-dimensional narrative embedding $\mathbf{e}_t$, the 6-dimensional attended signal vector $\mathbf{x}_t$, the 3-dimensional narrative regime encoding $\mathbf{n}_t$, and the 1-dimensional position state p_t). The branch consists of two fully connected layers with ReLU activations: $394 \to 128 \to 64$, producing the latent vector $\mathbf{z}_{\text{context}} \in \mathbb{R}^{64}$.

Price Branch (8,960 parameters): Processes technical indicators and fundamental quality. Input dimensionality is 10 (comprising the 7-dimensional technical indicator vector $\mathbf{f}_t$, the 2-dimensional fundamental quality encoding $\mathbf{q}_t$, and the 1-dimensional position state p_t, which is duplicated across both branches to enable position-aware feature extraction). The branch consists of two fully connected layers with ReLU activations: $10 \to 128 \to 64$, producing the latent vector $\mathbf{z}_{\text{price}} \in \mathbb{R}^{64}$.

Signal Attention & Context Gate (300 parameters): The attention mechanism (Equation 4.1) and gating network (Equation 4.2) together contribute 300 parameters, including the projection matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ and the gate MLP weights $\{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, b_2\}$.

Fusion & Q-Head (430 parameters): The branch outputs are fused via Equation 4.3 to produce $\mathbf{z}_{\text{final}} \in \mathbb{R}^{64}$. This is passed through a final Q-head consisting of two layers: $64 \to 64 \to 3$, outputting Q-values for the three actions $\{\text{BUY}, \text{HOLD}, \text{SELL}\}$.

The dual-stream design along with attention based fusion prevents the high-dimensional narrative embedding from dominating gradient updates during training, which would occur under naive early concatenation.

4.8 Training and Inference Strategy

4.8.1 Training Procedure

The agent is trained using DQN with experience replay [13]. At each time step, state transitions (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer $\mathcal{D}$ of fixed capacity (10,000 transitions). During each training iteration, a mini-batch of 64 transitions is sampled uniformly from $\mathcal{D}$ and the TD loss (Equation 4.11) is minimised via the Adam optimiser [21]:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}(\theta), \qquad (s, a, r, s') \sim \text{Uniform}(\mathcal{D}) \tag{4.13}$$

where $\eta = 5 \times 10^{-4}$ is the learning rate. Experience replay during training serves to break the temporal autocorrelation of consecutive training samples; without replay, the agent would train on highly correlated sequences (e.g., five consecutive days of a rally), violating the i.i.d. assumption underpinning stochastic gradient descent and causing the network to overfit to recent local patterns rather than learning generalisable policies. Also, it improves sample efficiency by enabling each transition to be reused multiple times across different gradient updates [22].

The target network parameters θ^- (used to compute the TD target in Equation 4.12) are updated every 1,000 training steps via a hard copy: $\theta^- \leftarrow \theta$. This periodic synchronisation stabilises training by preventing the regression target from shifting rapidly as the Q-network learns [13]. The discount factor hyperparameter has been set to $\gamma = 0.99$, reflecting a preference for long-term cumulative returns over myopic single-period gains.

Training proceeds for 50 epochs over the historical data, after which all network weights (including the attention mechanism, gating network, and Q-head) are frozen. All data prior to 5 January 2024 is used for training; all subsequent data is reserved for out-of-sample evaluation. This discipline is standard practice in quantitative finance [23] and guards against look-ahead bias (a form of data leakage that would arise from random shuffled splits). During training, the agent observes historical market scenarios and learns to associate state configurations (e.g., a bullish LLM narrative co-occurring with positive technical momentum and a flat position) with the BUY action, receiving reward feedback from subsequent price movements, and updating its weight iteratively, before being frozen for evluation on the out of sample dataset.

4.8.2 Inference and Deployment

During inference (5 January 2024 onwards), the agent operates in deployment mode: all network weights are frozen and no parameter updates occur. The complete historical dataset (including all transcripts, earnings data, and macro-economic indicators through the present) has already been ingested into the CTKG during the pre-processing phase; however, look-ahead bias is prevented through strict temporal filtering at query time.

For each weekly decision, the analytical pipeline executes as follows. The `PeerAnalystAgent` retrieves semantically similar Facts from peer companies via vector search, filtering to Facts associated with Quarters ending before the current date. The `HistoricalAnalystAgent` traverses the `[:NEXT_QUARTER]` temporal chain backwards from the most recent eligible Quarter, computing trend signals over the company's own historical Facts. The `SynthesizerAgent` aggregates these inputs alongside the time-appropriate EconomicContext node to generate the narrative and structured signals. The narrative is embedded, and the complete state vector $\mathbf{s}_t$ is constructed and passed to

the Q-network, which outputs Q-values for each feasible action.

The agent's decision logic is position-dependent, reflecting the long-only constraint (Equation 4.7). If the agent is currently flat ($p_t = 0$), it selects the action with highest Q-value from $\{\text{BUY}, \text{HOLD}\}$; a BUY decision transitions the agent to a fully long position ($p_{t+1} = 1$). If the agent is currently long ($p_t = 1$), it selects from $\{\text{SELL}, \text{HOLD}\}$; a SELL decision returns the agent to flat ($p_{t+1} = 0$). The HOLD action maintains the current position in both cases. Invalid actions are masked by setting their Q-values to $-\infty$ prior to the argmax operation.

Decisions are executed on a weekly rebalancing schedule (every Monday at market close). This cadence balances reducing sensitivity to high-frequency noise and minimising implicit transaction costs, whilst allowing technical indicators (which update daily) to provide intra-quarter signal updates between the sparse quarterly earnings announcements. The weekly schedule is consistent with the medium-term investment horizon implicit in the 300-word narrative format, and exploits the quarterly signals without letting them degenerate into noise for shorter rebalancing schedules.

Chapter 5

Backtesting and Results

This chapter reports the out-of-sample backtesting results for IntelliTrade over the period from 5 January 2024 to 2 February 2026. The evaluation compares the full system against an internal technical-only ablation, passive benchmarks, and selected external reference systems from the literature. The purpose of the chapter is to present the empirical behaviour of the end-to-end system and identify the main patterns in its performance across the evaluated assets in relation to baselines and benchmarks.

The analysis is fundamentally driven by questioning whether the full IntelliTrade system outperform returns and risk-adjusted performance against an otherwise identical DQN policy with all LLM-derived context removed. Additionally, how should these results be positioned relative to prior work in technical deep reinforcement learning for trading and more recent LLM-based trading agents.

5.1 Evaluation Methodology

5.1.1 Test Period and Market Context

The out-of-sample evaluation period provides a demanding environment for technology equities, covering strong AI-related momentum in semiconductor and infrastructure names, continued concentration in mega-cap technology, changing monetary-policy expectations, and episodes of elevated single-stock volatility. In particular, the period includes substantial upside (though with various downturn periods) in NVDA, ASML, TSM, and GOOGL, alongside more mixed and sentiment-sensitive behaviour in TSLA and META.

Particularly, this period is characterised by the broader AI investment cycle and elevated capital expenditure on data centers which benefited NVDA, ASML, and TSM; all major contributors to the AI infrastructure build out. GOOGL is also an important participator in this cycle, through its Gemini foundation models as well as its cloud and hardware stack (GCP and TPU), parallelising AMZN's role in providing AI infrastructure via AWS. TSLA and META, meanwhile, illustrate a more sentiment-sensitive profile. TSLA's equity story remains closely tied to progress in autonomous driving and AI hardware, whereas META's increased AI-related capital expenditure and product launches, such as Meta smart glasses, have so far translated into a more mixed perception of realised AI upside beyond its core advertising franchise, echoing investor concerns seen

in AAPL's incremental AI positioning. MSFT, despite strong and early investments in OpenAI as well as growth in Azure revenues, has been criticised by Wall Street for a perceived lack of clear, monetised AI differentiation relative to peers and for uncertainty around the incremental profitability of its AI-driven cloud and productivity offerings.

This market context lays the groundwork for interpreting the subsequent results, because the out of sample window combines exceptional AI infrastructure rallies driven by strong underlying technical numbers with volatile, narrative-driven stories which are not captured by quantitative methods alone, hence rendering the window and equity selection a strong test of IntelliTrade's capabilites.

5.1.2 Performance Metrics

Three classes of metrics are used.

Cumulative return is the primary profitability metric, computed as

$$\text{Cumulative Return} = \frac{V_{\text{end}} - V_{\text{start}}}{V_{\text{start}}} \times 100\%.$$

This measures total capital appreciation over the evaluation period. The results display both 'DQN Return' and 'Max DQN' where DQN return is the mean and standard deviation across all runs, and Max DQN is the single seed/run with the highest cumulative return.

Sharpe ratio is used to assess risk-adjusted return. This is important because higher raw return may otherwise reflect greater exposure to volatility rather than better trade selection.

Maximum drawdown (MDD) is reported to measure downside risk, capturing the largest peak-to-trough decline experienced during the backtest.

For each ticker, results are reported as mean $\pm$ standard deviation across five independent training runs. The standard deviation reflects variation arising from stochastic initialization and replay sampling. In addition, the maximum cumulative return across the five runs is reported to show best-case realized performance under different random seeds.

5.2 Internal Ablation Results

The main internal comparison is designed to isolate the contribution of contextual reasoning. Two variants were evaluated.

DQN (Full System): the complete IntelliTrade architecture, including CTKG retrieval, multi-agent LLM analysis, narrative embedding, signal attention, and gated fusion with the technical branch.

DQN No-LLM (Technical indicators only): an ablated version in which all context-derived features are masked and the context gate is forced to zero, leaving the policy to act only on price-derived technical indicators, fundamental quality encoding, and position state.

Tables 5.1 and 5.2 report cumulative return, Sharpe ratio, maximum drawdown, and best-run return for both variants, together with passive buy-and-hold returns for each ticker.

Table 5.1: Return-based backtest results for IntelliTrade variants and passive buy-and-hold over the out-of-sample period from 5 January 2024 to 2 February 2026.

Ticker	DQN Return	No-LLM Return	B&H	Max DQN	Max No-LLM
AAPL	43.09 ± 29.39	27.25 ± 19.54	50.4	79.1	48.6
AMZN	43.43 ± 27.12	28.43 ± 10.10	67.3	61.7	36.5
ASML	88.30 ± 38.26	24.26 ± 21.68	109.4	133.8	79.7
GOOGL	104.25 ± 18.66	79.74 ± 49.41	155.1	132.5	127.8
META	45.06 ± 18.61	14.13 ± 26.10	102.1	76.8	35.2
MSFT	-3.06 ± 11.56	-3.47 ± 15.54	16.8	8.6	-0.1
NVDA	111.01 ± 62.18	89.97 ± 89.30	278.3	190.5	165.7
TSLA	66.76 ± 63.43	32.11 ± 48.93	77.6	142.3	84.9
TSM	82.72 ± 40.27	87.17 ± 35.30	252.1	147.1	126.3
Mean	**64.62**	**42.18**	**123.23**	**108.04**	**78.29**

Table 5.2: Risk-based backtest results for IntelliTrade variants over the out-of-sample period from 5 January 2024 to 2 February 2026.

Ticker	DQN Sharpe	No-LLM Sharpe	DQN MDD	No-LLM MDD
AAPL	1.90 ± 1.05	1.38 ± 0.55	20.0 ± 11.0	22.2 ± 12.7
AMZN	1.90 ± 1.02	1.40 ± 0.53	21.1 ± 9.5	24.0 ± 5.9
ASML	1.45 ± 0.28	1.10 ± 0.64	32.3 ± 13.8	31.8 ± 13.0
GOOGL	3.37 ± 0.51	2.49 ± 1.30	22.8 ± 6.7	28.4 ± 11.1
META	1.71 ± 0.49	0.41 ± 1.56	33.5 ± 1.1	32.6 ± 1.7
MSFT	0.03 ± 0.63	0.16 ± 1.08	21.4 ± 1.6	21.9 ± 8.4
NVDA	2.49 ± 0.98	2.08 ± 1.72	27.1 ± 7.7	27.2 ± 11.7
TSLA	1.52 ± 1.05	0.53 ± 1.55	33.3 ± 6.2	38.9 ± 8.5
TSM	2.78 ± 0.78	2.67 ± 0.60	24.0 ± 6.3	27.6 ± 7.2
Mean	**1.91**	**1.47**	**26.17**	**27.63**

5.2.1 Aggregate Performance

The full IntelliTrade system achieved a mean cumulative return of 64.62% across the nine equities, compared with 42.18% for the technical-only ablation. This corresponds to an average improvement of 22.44 percentage points, when including LLM generated signals. The full system outperformed the no-LLM baseline on eight of the nine tickers in mean returns, with TSM the only exception.

Because the reinforcement learning architecture is otherwise unchanged, this comparison isolates the contribution of contextual information. The observed uplift is therefore consistent with the view that transcript-derived context used in the mulit-agent LLM signal generation contributes useful information beyond price and technical indicators alone.

The largest improvements were observed on ASML (+64.04 percentage points), TSLA (+34.65), META (+30.93), and GOOGL (+24.51). These are all cases where management guidance, sector narratives, competitive positioning, or supply-chain commentary may matter alongside technical structure. By contrast, MSFT showed almost no difference between variants, and TSM slightly favoured the technical-only model. Within this evaluation setting, the contextual branch appears beneficial on average, but not uniformly across all assets.

These patterns are consistent with the earlier market context where the contextual branch is most helpful precisely on those names where AI based narratives, management guidance, and competitive positioning played a central role in performance (ASML, TSLA, META, GOOGL), while its impact is muted, or even negative, in cases where returns were dominates either by exceptionally strong underlying trends (NVDA, TSM) or where the AI story remains less clearly differentiated (MSFT).

5.2.2 Risk-Adjusted Performance

Risk-based results in Table 5.2 allow the comparison to go beyond cumulative return. On average, the full IntelliTrade system achieved a mean Sharpe ratio of 1.91, compared with 1.47 for the technical-only ablation. This suggests that the contextual branch improved return per unit of risk as well as raw profitability.

This pattern is especially visible on GOOGL, where Sharpe rose from 2.49 to 3.37, and on META, where it rose from 0.41 to 1.71. TSLA also showed a notable improvement, from 0.53 to 1.52. These cases suggest that the contextual branch may improve trade timing or position quality rather than simply increasing aggressiveness.

Maximum drawdown results are more mixed. On some tickers the full system reduced downside risk relative to the ablation, for example AAPL (20.0 vs. 22.2), AMZN (21.1 vs. 24.0), GOOGL (22.8 vs. 28.4), and TSLA (33.3 vs. 38.9). On others, especially ASML and META, drawdown was similar or slightly higher. The main pattern is therefore stronger and more consistent in return and Sharpe than in drawdown reduction.

5.2.3 Variance Across Runs

The standard deviations across five training seeds show that performance remains sensitive to stochastic training dynamics, particularly on the most volatile names. NVDA exhibits high variance under both variants, with standard deviations of 62.18 and 89.30 respectively, while TSLA also shows substantial variance for both models. This is unsurprising given the strong momentum and sentiment sensitivity of both stocks over the evaluation window.

Even so, the contextual branch sometimes reduces instability. On GOOGL, the full system not only improves mean return but also reduces the return standard deviation from 49.41 to 18.66. On META, return increases sharply while variance also falls relative to the no-LLM model. These cases suggest that contextual features may help the policy converge toward more stable trading behaviour for certain assets.

5.3 Comparison with Passive Baselines

5.3.1 Buy-and-Hold of Individual Tickers

The buy-and-hold comparison clarifies what the internal ablation does and does not establish. IntelliTrade clearly outperformed the technical-only active baseline on average, but it did *not* outperform passive buy-and-hold on average across the selected technology stocks. The mean buy-and-hold return across the nine tickers was 123.23%, substantially above the full system's mean return of 64.62%.

The maximum-run results provide a more optimistic but narrower view. The best case seed full-system run exceeded buy-and-hold on three of the nine tickers: AAPL (79.1% vs. 50.4%), ASML (133.8% vs. 109.4%), and TSLA (142.3% vs. 77.6%). This shows that the architecture can outperform passive exposure on selected assets in favourable realizations, even though that is not the average outcome.

At the same time, the mean of the maximum full-system runs (Max DQN) was 108.04%, which still remained below the average buy-and-hold return of 123.23%. Hence, while IntelliTrade can

beat buy-and-hold on selected names in its strongest runs, the combination of performance variance across seeds and missed upside on extreme momentum stocks prevents it from matching the average passive return over this window.

5.3.2 Comparison with the Broad Market

A broader passive comparison is also useful; using the S&P 500 benchmark return of 45.54% over the same period, from 5 January 2024 to 2 February 2026, IntelliTrade's mean return of 64.62% exceeds the broad market benchmark by 19.08 percentage points.

This is a more balanced comparison than averaging passive returns across a deliberately selected set of strong technology stocks. The nine evaluation names are concentrated in one of the strongest market segments of the period, so their average buy-and-hold return forms a particularly demanding hurdle. Relative to the market, IntelliTrade's active performance over its universe is more favourable.

5.4 Comparison with External Work

5.4.1 Comparison with Zhang et al.

Zhang et al. proposed deep reinforcement learning methods for trading 50 liquid futures contracts from 2011 to 2019 and reported that their RL strategies outperformed classical time-series momentum baselines while remaining profitable under substantial transaction costs [14]. Their setup differs from IntelliTrade in asset class, reward design, and evaluation period, so direct numerical comparison is not appropriate.

The comparison is therefore architectural rather than numerical. Zhang et al. showed that DQN-style policies can extract useful signal from price-based features alone in a trading setting. IntelliTrade builds on that idea by asking whether price-based RL can be improved through structured contextual enrichment. The ablation results reported here are consistent with that conclusion, where adding CTKG retrieval and LLM-derived features improved both mean cumulative return and mean Sharpe ratio relative to the technical-only DQN.

5.4.2 Comparison with TradingAgents

TradingAgents by TauricResearch is a directly relevant external reference, one which has influenced this project as mentioned in preceding sections. TradingAgents employs a multiagent architecture which includes a Market Analyst, Social Media Analyst, News Analyst, and Fundamental Analyst, with three 'bullish', 'bearish', and 'neutral' agents for each analyst class respectively. The canonical use of TradingAgents involves selecting a stock ticker and analysis date from which the agents debate, provide comprehensive reports alongside a final analysis decision from one of the following classes: 'sell', 'overweight', 'hold', 'underweight', and 'buy'.

To simulate an equivalent trading environment to IntelliTrade, a script has been written by using the TradingAgents API [37] in Python to forward propagate decisions on all tickers for the same period and weekly rebalancing schedule when backtesting IntelliTrade, as well as being given an initial capital of $10000 and maintaining buy-only trade masking. For consistency with IntelliTrade, 'overweight', and 'underweight' decisions from TradingAgents are simulated as sell and buy respectively, and the same large language model, GPT 5.2 through Azure Foundry was used. Due

to the long runtime for a single decision (4-5 minutes) and over 100 trading weeks to simulate over 9 tickers, a Linux Virtual Machine on Azure was deployed to run the experiment throughout multiple days.

The following results have been obtained through the simulation, where TA denotes TradingAgents, and IT denotes IntelliTrade:

Table 5.3: TradingAgents Simulation Results by Ticker

Ticker	**TA Final Price ($)**	**TA Return (%)**	**IT Max DQN**	**IT Mean DQN**
TSM	22634.7817	126.3478	147.1	82.72
ASML	15592.9433	55.9294	133.8	88.30
MSFT	10189.4512	1.8945	8.6	-3.06
AMZN	15709.2936	57.0929	61.7	43.43
GOOGL	24275.8978	142.7590	132.5	104.25
AAPL	11881.7658	18.8177	79.1	43.09
META	17536.9401	75.3694	76.8	45.06
NVDA	7502.8475	-24.9715	190.5	111.01
TSLA	10000.0000	0.0000	142.3	66.76
Mean		50.3599	108.04	64.62

The mean return for the TradingAgents simulation across all tickers was 50.36, underperforming IntelliTrade's sampled mean return of 64.62, indicating that overall, IntelliTrade is able to beat the state of the art multi agent trading framework. Additionally, IntelliTrade's best seed case, 'Max DQN', outperforms the TradingAgents results in all but one ticker (GOOGL). However, a consequence of IntelliTrade's high variance leads to weaker performance relative to TradingAgents in the mean (average of seeds) case, where IntelliTrade generates a higher return only for META, NVDA, and TSLA.

Therefore, these results suggest that IntelliTrade can match or exceed the TradingAgents framework on the evaluated equity universe in favourable seed realizations and exhibits positive average alpha across seeds and tickers, but this advantage is partially offset by higher variance across runs.

5.4.3 Comparison with Public LLM Trading Leaderboards

A more contemporary, though still imperfect, external reference point is Nof1's *Alpha Arena*, a public competition-style trading environment for AI models. Nof1 states that the official competition ended on 3 December 2025, after which models were no longer running, and reports that the winning "Mystery Model" achieved a 12.11% return over the competition period [42]. The public leaderboard further reports model rankings by total equity, providing a live-style evaluation setting with initial capital of $10000, a similar setup to IntelliTrade [42].

By comparison, IntelliTrade achieved a mean cumulative return of 64.62% and a mean Sharpe ratio of 1.91 over its own out-of-sample evaluation window. On these summary metrics, IntelliTrade compares favourably with the uploaded Alpha Arena results.

However, this comparison should be interpreted cautiously as Alpha Arena uses a different asset universe, trading environment, competition protocol, and evaluation horizon. It is therefore more useful as a contextual reference showing that LLM-based trading systems are now being evaluated in public competition-style settings than as a strict like-for-like benchmark for IntelliTrade.

5.5 Qualitative Case Studies

In addition to ticker-level aggregate metrics, this section includes a trade-level example to illustrate how the learned policy behaves within a single out-of-sample run.

5.5.1 ASML: Strongest Gain from Context Enrichment

ASML shows the largest improvement from contextual enrichment, rising from 24.26% under the technical-only baseline to 88.30% under the full system. This is one of the clearest cases in which the contextual branch appears to add useful signal. ASML's valuation over the evaluation window was heavily shaped by AI infrastructure demand, semiconductor capital expenditure expectations, geopolitical export restrictions, and management commentary on backlog and customer demand. These are the kinds of signals that a transcript-aware contextual system may capture better than a purely technical state representation.

5.5.2 TSM: A Useful Counterexample

TSM is the only ticker for which the no-LLM model outperforms the full system, making it a useful counterexample rather than a result to omit. One plausible interpretation is that TSM's strong uptrend was sufficiently pronounced that a simpler technical policy could exploit it effectively, while the contextual branch added little incremental signal over price-based features alone. Regardless of the precise cause, TSM shows that contextual enrichment is not uniformly beneficial.

5.5.3 MSFT: Minimal Signal from Either Variant

MSFT performs poorly under both variants, with slightly negative cumulative returns and near-zero Sharpe for the full model. The difference between the two systems is negligible. Within this evaluation setting, MSFT appears to be a case where transcript-derived context added little useful information at the weekly trading horizon considered here, likely due to noisy sentiment around AI capex as discussed earlier.

5.5.4 NVDA: Example Trade Trajectory

Figure 5.1 provides a concrete example of the full IntelliTrade system's behaviour on NVDA in a single out-of-sample run. NVDA is a particularly informative case because it was one of the strongest-performing and most volatile equities in the evaluation universe. This makes it a difficult asset for active trading: a successful policy must remain exposed to major upward trends without being excessively damaged by intermediate corrections or sentiment-driven reversals.

Several aspects of the trajectory and buying behavior are noteworthy. First, the policy captures a substantial fraction of NVDA's broader upward move, ending with a cumulative return of 144.8% and a final portfolio value of $24,478 from a $10,000 initial allocation. Second, the sequence of buy and sell markers suggests that the policy does not simply hold continuously; instead, it re-enters during renewed momentum and exits around weaker or more uncertain periods. In other words, the learned policy appears to exploit large directional moves while avoiding full participation in every drawdown.

This behaviour is relevant from a risk-adjusted perspective. A passive buy-and-hold strategy may still achieve a higher raw return on a stock such as NVDA during a strongly trending regime, but it



Figure 5.1: Example IntelliTrade trading trajectory on NVDA for a single full-system run over the out-of-sample period from 5 January 2024 to 2 February 2026. Green upward triangles indicate buy actions and red downward triangles indicate sell actions. The run achieved a cumulative return of 144.8%, final portfolio value of $24,478, maximum drawdown of 23.3%, Sharpe ratio of 1.38, and a win rate of 73% across 15 completed trades.

must also absorb the full path volatility of that move. By contrast, an active policy that captures much of the upside while reducing exposure during adverse swings may achieve stronger risk-adjusted performance even when its absolute return remains below the passive benchmark. The example in Figure 5.1 is consistent with that interpretation. The trade-level statistics displayed in Figure 5.2 also provide useful qualitative evidence about policy behaviour. The run executed 15 buys and 15 sells, with 11 profitable completed trades, corresponding to a win rate of 73%. This suggests that the result does not depend on one isolated successful trade; instead, performance is distributed across repeated entries and exits over the evaluation window. The figure therefore complements the aggregate tables by illustrating how the contextual DQN can produce improved outcomes in practice, particularly by minimising risk for highly volatile stocks while capturing huge upside.

5.6 Experimental Limitations

These results should be interpreted with several experimental limitations in mind.

Single market regime. The evaluation period from 5 January 2024 to 2 February 2026 captures only one broad market regime, albeit a long and eventful one. A longer multi-cycle backtest would provide a stronger test of robustness.

Small and homogeneous universe. The nine evaluated equities are all large-cap technology stocks. This is suitable for testing the dissertation's core hypothesis, but it limits generalisation to other sectors and asset classes.

No transaction cost modelling. Returns are reported gross of slippage, spreads, and commis-

Date	Action	Price	Shares	Value	Portfolio
2024-02-26	BUY	$174.73	57	$9,959.61	$9,990.04
2024-03-04	SELL	$177.58	57	$10,122.06	$10,142.37
2024-03-11	BUY	$171.96	58	$9,973.68	$10,132.39
2024-03-18	SELL	$174.48	58	$10,119.84	$10,268.43
2024-05-06	BUY	$188.70	54	$10,189.80	$10,258.25
2024-08-05	SELL	$161.02	54	$8,695.08	$8,754.83
2024-08-12	BUY	$166.80	52	$8,673.60	$8,746.16
2025-05-27	SELL	$206.02	52	$10,713.04	$10,774.88
2025-06-16	BUY	$216.10	49	$10,588.90	$10,764.29
2025-11-10	SELL	$248.40	49	$12,171.60	$12,334.82
2025-11-24	BUY	$226.28	54	$12,219.12	$12,322.60
2025-12-01	SELL	$233.88	54	$12,629.52	$12,720.37
2025-12-08	BUY	$226.89	56	$12,705.84	$12,707.67
2026-01-26	SELL	$238.42	56	$13,351.52	$13,340.00
2026-02-02	BUY	$242.96	54	$13,119.84	$13,326.88

Figure 5.2: Trade-level statistics for an example trading trajectory on NVDA or a single seed full-system run over the out-of-sample period

sions. Weekly trading reduces this concern relative to higher-frequency systems, but it does not eliminate it.

High seed sensitivity on some names. The substantial standard deviations on NVDA and TSLA indicate that the learned policies remain sensitive to training stochasticity. Formal significance testing would strengthen the statistical interpretation of the observed differences.

5.7 Summary

This chapter reported the out-of-sample backtesting results for IntelliTrade. The full system outperformed the technical-only ablation on eight of the nine evaluated equities, increasing mean cumulative return from 42.18% to 64.62% and mean Sharpe ratio from 1.47 to 1.91. Passive buy-and-hold remained stronger on average across the selected technology basket, although IntelliTrade exceeded the broad market benchmark and outperformed buy-and-hold on selected tickers in its strongest runs.

Chapter 6

Conclusions and Evaluation

This dissertation investigated the following research question:

> *Can grounding a DRL trading agent in semantically rich context, extracted and structured by specialized LLM agents querying a causal-temporal knowledge graph, improve risk-adjusted trading performance relative to baseline strategies that rely on either technical analysis alone or passive benchmarks?*

The project answered this question by designing, implementing, and evaluating IntelliTrade: a hybrid trading system that combines structured transcript retrieval through a Causal-Temporal Knowledge Graph (CTKG), multi-agent LLM signal generation, and a context-aware Deep Q-Network (DQN). The results show that, within the evaluation setting used in this dissertation, context enrichment improved performance relative to the technical-only reinforcement learning baseline. Across the nine evaluated technology equities, the full system increased mean cumulative return from 42.18% to 64.62% and mean Sharpe ratio from 1.47 to 1.91, outperforming the no-LLM ablation on eight of nine tickers. Additionally, comparison with the state-of-the-art multi-agent trading framework, TradingAgents, shows that IntelliTrade can match or exceed its performance on the evaluated equity universe in favourable realizations while delivering positive average alpha over the tested tickers.

6.1 Achievement of Project Goals

The project set out to deliver five specific outputs. These were achieved as follows.

6.1.1 Goal 1: Multi-Modal Financial Data Pipeline

The first goal was to build a data pipeline aggregating daily OHLCV price histories, quarterly earnings transcripts, fundamental metrics, and macroeconomic indicators for nine technology equities, with temporal alignment supporting walk-forward validation. Chapter 4 described the end-to-end collection and preprocessing pipeline, including the extraction of technical indicators from price data, integration of quarterly fundamentals, incorporation of macroeconomic context, and alignment of transcript-derived information with the market timeline.

This was an important foundation for the rest of the project. Without reliable temporal alignment and comprehensive data pre-processing and aggregation, later stages of the system would have

been vulnerable to leakage or inconsistent state construction. The resulting dataset was sufficient to support training on pre-2024 data and out-of-sample testing from 5 January 2024 onwards.

6.1.2 Goal 2: Causal-Temporal Knowledge Graph

The second goal was to construct a Causal-Temporal Knowledge Graph in Neo4j to encode causal and temporal relationships extracted from earnings transcripts. The implemented graph schema represented entities, events, and relationships in a form that supported both structured traversal and vector-based semantic retrieval by the LLM agents using Neo4j Cypher queries on a cloud based Aura DB.

This design was justified by the requirements analysis. A flat vector store would have provided semantic similarity but weak temporal control, while a relational design would have struggled to support flexible semantic retrieval. The graph-based approach enabled the system to preserve causal and temporal structure while still supporting embedding-based search, which was central to the retrieval needs of the agent pipeline.

6.1.3 Goal 3: Multi-Agent LLM Signal Generation

The third goal was to implement a four-agent LLM pipeline that transformed unstructured earnings transcripts into structured signals and natural-language narratives. This goal was achieved through the development of specialised agents for fact extraction, peer analysis, historical analysis, and synthesis.

This was one of the most distinctive parts of the project. Rather than relying on a single monolithic prompt, the system decomposed the reasoning process into modular agentic stages. This improved interpretability and aligned with the analytical requirements identified earlier in the dissertation. The output of these agents was then embedded and passed into the downstream trading architecture, making the contextual reasoning operational rather than merely descriptive.

6.1.4 Goal 4: Context-Aware Deep Q-Network

The fourth goal was to build a context-aware DQN with dual-stream architecture, learned attention, and gating mechanisms that fused LLM-derived context with technical indicators. The final system implemented a dual-stream network with separate pathways for contextual and price-based features, an attention mechanism for signal weighting, and a gating mechanism for learned fusion.

This architecture was important because it avoided two undesirable extremes. It did not ignore context entirely, as a technical-only policy would, but it also did not allow the contextual branch to dominate the state representation indiscriminately. Instead, the learned gate provided a mechanism through which the system could adjust how much contextual information influenced the final policy.

6.1.5 Goal 5: Empirical Evaluation Against Baselines

The fifth goal was to conduct empirical evaluation against baseline strategies on out-of-sample data, demonstrating whether context enrichment improved risk-adjusted performance. This goal was achieved through the backtesting and ablation studies reported in Chapter 5.

The internal evaluation showed that the full system outperformed the technical-only DQN on eight of nine equities, with higher mean return and higher mean Sharpe ratio. Comparisons were also

made against passive buy-and-hold, a broad-market benchmark, and external reference systems from the literature. The cleanest evidence came from the internal ablation: because the technical-only and full systems shared the same reinforcement learning framework, the uplift provides reasonably direct evidence that contextual signals contributed useful information. The comparison against passive baselines was more mixed. IntelliTrade exceeded the S&P 500 benchmark over the comparison window, but did not outperform average buy-and-hold across the selected technology basket.

Overall, this means that the project's empirical evaluation objective was met, and that the dissertation can answer its central research question with a qualified and quantified yes: contextual enrichment improved risk-adjusted performance relative to the technical-only DRL baseline, but did not universally dominate passive ownership of the strongest individual technology names.

6.2 Critical Evaluation of the Project

The project was successful in meeting its main technical and research objectives. A complete end-to-end system was implemented, integrating heterogeneous financial data, structured knowledge representation, LLM-based reasoning, and reinforcement learning. This alone represents a substantial achievement, because the dissertation did not stop at conceptual design; it delivered a functioning research prototype and subjected it to empirical evaluation.

The strongest aspect of the project is the coherence between the research question, system design, and evaluation methodology. The design choices made in Chapters 3 and 4 were directly reflected in the backtesting results in Chapter 5. The CTKG addressed the need for structured temporal retrieval. The multi-agent architecture addressed the need for decomposed reasoning. The dual-stream DQN addressed the need to combine context with technical state without collapsing one into the other. This alignment between requirements, implementation, and results strengthens the credibility of the dissertation as a whole.

A second strength is that the project did not rely on a weak or unfair comparison. The internal technical-only DQN baseline was a meaningful ablation that observed uplift from 42.18% to 64.62% in mean cumulative return, and from 1.47 to 1.91 in mean Sharpe ratio. This provided useful, empirical evidence that the contextual pipeline added value beyond price-based technical analysis alone. Furthermore, an equivalent trading environment was simulated over TradingAgents for an equitable comparison. Despite this, TradingAgents was only simulated for a single run due to the expensive API costs and long runtimes, whereas an ideal comparison would evaluate multiple independent seeds for both systems.

However, the project also has important limitations. The most significant is variance across runs: while some seeds achieve very strong outcomes on individual tickers, others materially underperform, which complicates interpretation of headline results. A key takeaway from the behaviour of the system is its ability to capture upside while avoiding prolonged downturns, which reduces exposure to risk and helps deliver strong risk-adjusted performance despite remaining below passive benchmarks on average.

Another limitation is evaluation scope. The universe consisted of only nine large-cap technology equities, and the out-of-sample period, although reasonably long, still captures only one broad market regime. This constrains the degree to which the findings can be generalized. The absence of transaction cost modelling is also important, because active strategies are particularly sensitive to

trading frictions. While weekly rebalancing reduces the severity of this issue, it does not eliminate it.

There are also limitations at the implementation level. The system is complex, involving graph storage, retrieval, multiple LLM stages, embedding generation, and downstream policy learning. This complexity is justified by the gains observed in the ablation study, but it also introduces practical overheads that would matter in real deployment. Similarly, some names such as NVDA and TSLA showed high variance across seeds, suggesting that training stability remains an open challenge.

In summary, the project can be judged as successful as a research prototype and dissertation contribution. It is fit for the purpose it set itself: to test whether structured LLM-derived context can improve DRL trading decisions. It should not yet be considered a deployment-ready trading platform, nor does it show that active contextual trading universally beats passive strategies. The evidence is promising, but bounded.

6.3 Evaluation of Personal Aims

The project also met the personal aims stated in the introduction.

First, it provided substantial experience with transformer-style attention mechanisms in two settings: understanding LLM-based text processing and implementing learned attention within the trading architecture itself.

Second, it offered practical experience in reinforcement learning environment design, reward formulation, and applying DRL in a non-stationary financial setting with delayed rewards. This directly supports the personal goal of building intuition relevant to post-training reinforcement learning more broadly.

Third, it developed hands-on experience with graph databases, including Neo4j schema design, Cypher querying, and temporal knowledge representation.

Fourth, it required the integration of multiple subsystems, including LLMs, knowledge graphs, neural networks, and financial data pipelines, into one cohesive end-to-end system. This was one of the most educationally valuable aspects of the project, because it required not only isolated model building but system-level engineering.

Finally, the project provided a realistic view of the opportunities and limits of AI in finance. It showed that machine learning can exploit meaningful structure in financial text, but also that real-world trading problems remain difficult, regime-dependent, and highly sensitive to evaluation design.

6.4 Future Work

Several extensions would strengthen and broaden the present system.

First, the evaluation could be expanded substantially. A larger and more diverse asset universe would help determine whether the observed gains generalize beyond large-cap technology equities. Testing across additional sectors, more market regimes, and longer historical windows would provide a stronger assessment of robustness and reduce the risk that the current findings are overly

specific to the selected universe and evaluation period. The evaluation could also be deepened through more extensive statistical analysis and richer performance attribution, enabling a clearer understanding of when and why the contextual branch contributes most.

Second, the contextual data sources could be broadened beyond quarterly earnings transcripts. While transcripts provide high-value management commentary, they are relatively sparse and event-driven. Incorporating financial news, analyst reports, guidance revisions, and macroeconomic event summaries would allow the system to capture a denser and more continuously updated information environment. This would make the contextual branch more responsive between earnings releases and provide a stronger test of whether structured language-based signals remain useful at higher update frequencies.

Third, a more ambitious extension would be to move beyond using the language model system as a frozen feature generator and instead apply reinforcement learning post-training to open-source models themselves. In such a setup, the model weights would be updated directly based on downstream trading performance, rather than keeping the language model fixed and only consuming its outputs through a separate downstream policy. This would make it possible to test whether a post-trained model can learn to produce progressively more decision-useful financial reasoning over time. An especially interesting variant would be to remove the separate fusion model entirely and use only the model's own outputs as the trading signal. This would shift the project from a context-enriched RL architecture toward an RL-adapted language model for trading, and would provide a natural continuation of the ideas explored in this dissertation.

6.5 Final Remarks

This dissertation set out to investigate whether semantically rich contextual reasoning could improve deep reinforcement learning for trading. The result is a qualified but meaningful positive answer. Within the evaluated setting, grounding a DRL agent in structured, LLM-derived context improved both return and risk-adjusted performance relative to a technical-only baseline.

More broadly, the project demonstrated that symbolic structure, natural-language reasoning, and sequential decision learning can be combined into a single financial AI system. Even where the final system fell short of passive buy-and-hold on the strongest individual technology names, it still showed that contextual enrichment is not merely a theoretical idea: it can be implemented end-to-end and can produce measurable gains.

The most important contribution of the project is therefore not the construction of a universally superior trading strategy, but the demonstration that CTKG-guided, multi-agent contextual reasoning is a viable and empirically promising enhancement to reinforcement learning in finance. That is a useful result both for this specific trading problem and for the broader question of how language-based reasoning can be integrated into decision-making systems.

Bibliography

[1] Precedence Research, "Algorithmic trading market size, share & trends analysis report 2025–2034," Tech. Rep., Jan. 2025. [Online]. Available: https://www.precedenceresearch.com

[2] Grand View Research, "High frequency trading market size & share report, 2030," Tech. Rep., 2024. [Online]. Available: https://www.grandviewresearch.com

[3] R. S. Tsay, *Analysis of Financial Time Series*, 2nd ed. Hoboken, NJ: Wiley, 2005.

[4] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, Oct. 2018. doi: 10.1016/j.ejor.2017.11.054.

[5] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, Mar. 2018.

[6] Y. Li, W. Zheng, and Z. Zheng, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[7] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, New York, NY, Jun. 2016, pp. 1928–1937.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, Jul. 2017.

[9] T. Théate and D. Ernst, "An application of deep reinforcement learning to algorithmic trading," *Expert Systems with Applications*, vol. 173, p. 114632, Jul. 2021. doi: 10.1016/j.eswa.2021.114632.

[10] Z. Yang et al., "FinGPT: Open-source financial large language models," *arXiv preprint arXiv:2306.06031*, Jun. 2023.

[11] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

[12] Li, C. (2024). Context enriched agentic RAG: Integrating causal-temporal knowledge graphs with large language model agents for financial analysis. *Unpublished technical report.*

[13] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.

[14] Zhang, Z., Zohren, S., and Roberts, S. (2020). Deep reinforcement learning for trading. *The*

Journal of Financial Data Science, 2(3):25–40. Oxford-Man Institute of Quantitative Finance, University of Oxford. Preprint: `arXiv:1911.10107`.

[15] A. Bigeard, L. Nashold, R. Krishnan, and S. Wu, "Finance Agent Benchmark: Benchmarking LLMs on real-world financial research tasks," *arXiv preprint arXiv:2508.00828*, May 2025. [Online]. Available: https://arxiv.org/abs/2508.00828

[16] N. Gupta, R. Chatterjee, H. Oiwa *et al.*, "DeepSearchQA: Bridging the comprehensiveness gap for deep research agents," *arXiv preprint arXiv:2601.20975*, Jan. 2026. [Online]. Available: https://www.kaggle.com/benchmarks/google/dsqa

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Language Process. (EMNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[18] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Proc. Conf. Empirical Methods Natural Language Process. (EMNLP)*, Abu Dhabi, UAE, Dec. 2022, pp. 11370–11384. [Online]. Available: https://arxiv.org/abs/2210.07316

[19] J. Arevalo, T. Solorio, M. Montes-y-Gómez, and F. A. González, "Gated multimodal units for information fusion," in *Proc. 5th Int. Conf. Learning Representations (ICLR) Workshop*, Toulon, France, Apr. 2017. [Online]. Available: https://arxiv.org/abs/1702.01992

[20] J. W. Wilder Jr., *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research, 1978.

[21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, May 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[22] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, no. 3, pp. 293–321, May 1992. doi: 10.1007/BF00992699.

[23] R. Pardo, *The Evaluation and Optimization of Trading Strategies*, 2nd ed. Hoboken, NJ: Wiley, 2008.

[24] T. Loughran and B. McDonald, "When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks," *The Journal of Finance*, vol. 66, no. 1, pp. 35–65, Feb. 2011. doi: 10.1111/j.1540-6261.2010.01625.x.

[25] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, Aug. 2019.

[26] Y. Yang, M. C. S. Uy, and A. Huang, "FinBERT: A pretrained language model for financial communications," *arXiv preprint arXiv:2006.08097*, Jun. 2020.

[27] S. Wu et al., "BloombergGPT: A large language model for finance," *arXiv preprint arXiv:2303.17564*, Mar. 2023.

[28] H. Yang, X.-Y. Liu, and C. D. Wang, "FinGPT: Open-source financial large language models," *arXiv preprint arXiv:2306.06031*, Jun. 2023.

[29] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. 10th Int. Conf. Learning Representations (ICLR)*, Apr. 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[30] Z. Xie et al., "FinChain: A symbolic benchmark for verifiable chain-of-thought financial reasoning," *arXiv preprint arXiv:2506.02515*, Jun. 2025.

[31] Nof1.AI, "Exploring the limits of large language models as financial trading agents," Tech. Blog, Jan. 2025. [Online]. Available: https://nof1.ai/blog/TechPost1

[32] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artificial Intelligence (AAAI)*, Phoenix, AZ, Feb. 2016, pp. 2094–2100.

[33] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, New York, NY, Jun. 2016, pp. 1995–2003.

[34] T. Théate and D. Ernst, "An application of deep reinforcement learning to algorithmic trading," *Expert Systems with Applications*, vol. 173, p. 114632, Jul. 2021. doi: 10.1016/j.eswa.2021.114632.

[35] X.-Y. Liu, H. Yang, J. Gao, and C. D. Wang, "FinRL: Deep reinforcement learning framework to automate trading in quantitative finance," in *Proc. ACM Int. Conf. AI in Finance (ICAIF)*, Nov. 2021. [Online]. Available: https://arxiv.org/abs/2011.09607

[36] Y. Xiao, E. Sun, D. Luo, and W. Wang, "TradingAgents: Multi-agents LLM financial trading framework," *arXiv preprint arXiv:2412.20138*, presented at AAAI 2025 Workshop Multi-Agent AI in the Real World, Dec. 2024.

[37] Tauric Research, "TradingAgents: Multi-Agents LLM Financial Trading Framework," GitHub repository, 2024. [Online]. Available: https://github.com/TauricResearch/TradingAgents

[38] X. V. Li and F. S. Passino, "FinDKG: Dynamic knowledge graphs with large language models for detecting global trends in financial markets," in *Proc. ACM Int. Conf. AI in Finance (ICAIF)*, Nov. 2024. [Online]. Available: https://arxiv.org/abs/2407.10909

[39] L. Shi, Z. Wang, Y. Zhang, and S. Ke, "Integrated GCN-LSTM stock prices movement prediction based on knowledge-incorporated graphs construction," *International Journal of Machine Learning and Cybernetics*, vol. 14, pp. 2497–2512, 2023. doi: 10.1007/s13042-023-01779-4.

[40] C. Li, "Context-enriched agentic RAG: Integrating causal-temporal knowledge graphs with large language model agents for financial analysis," Unpublished technical report, 2024.

[41] S. Li, Y. Xie, X. Chen, and R. Yan, "CausalStock: Deep end-to-end causal discovery for news-driven multi-stock movement prediction," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2411.06391

[42] Nof1, "Alpha Arena Leaderboard," 2026. [Online]. Available: `https://nof1.ai/leaderboard`. [Accessed: 29-Mar-2026].

Appendix A

NVIDIA Q2 Fiscal 2026 Earnings Call Transcript Excerpt

Operator: Good afternoon. My name is Sarah, and I will be your conference operator today. At this time, I would like to welcome everyone to NVIDIA Corporation's Second Quarter Fiscal 2026 Financial Results Conference Call. All lines have been placed on mute to prevent any background noise. After the speakers' remarks, there will be a question and answer session. If you would like to ask a question during this time, simply press star followed by the number one on your telephone keypad. If you would like to withdraw your question, press star 1 again. Thank you. Toshiya Hari, you may begin your conference. Thank you.

Operator: Good afternoon, everyone, and welcome to NVIDIA Corporation's conference call for 2026. With me today from NVIDIA Corporation are Jensen Huang, president and chief executive officer, and Colette Kress, executive vice president and chief financial officer. I would like to remind you that our call is being webcast live on NVIDIA Corporation's site. The content of today's call is NVIDIA Corporation's property. It cannot be reproduced or transcribed without our prior written consent. During this call, we may make forward-looking statements based on current expectations. These are subject to a number of significant risks and uncertainties, and our actual results may differ materially. For a discussion of factors that could affect our future financial results and business, please refer to the disclosure in today's earnings release, our most recent forms 10-K and 10-Q, and the reports that we may file on Form 8-K with the Securities and Exchange Commission. All our statements are made as of today, 08/27/2025, based on information currently available to us. Except as required by law, we assume no obligation to update any such statements. During this call, we will discuss non-GAAP financial measures. You can find a reconciliation of these non-GAAP financial measures to GAAP financial measures in our CFO commentary, which is posted on our website. With that, let me turn the call over to Colette.

Colette Kress (CFO): We delivered another record quarter while navigating what continues to be a dynamic external environment. Total revenue was $46.7 billion, exceeding our outlook as we grew sequentially across all market platforms. Data center revenue grew 56% year over year. Data center revenue also grew sequentially despite the $4 billion decline in H20 revenue. NVIDIA Corporation's Blackwell platform reached record levels, growing sequentially by 17%. We began production shipments of GB300 in Q2. Our full stack AI solutions for cloud service providers, Neo

Clouds, enterprises, and sovereigns are all contributing to our growth.

We are at the beginning of an industrial revolution that will transform every industry. We see $3 to $4 trillion in AI infrastructure spend by the end of the decade. The scale and scope of these build-outs present significant long-term growth opportunities for NVIDIA Corporation. The GB200 NBL system is seeing widespread adoption with deployments at CSPs and consumer Internet companies. Lighthouse model builders, including OpenAI, Meta, and Mastral, are using the GB200 NBL72 at data center scale for both training next-generation models and serving inference models in production.

The new Blackwell Ultra platform has also had a strong quarter, generating tens of billions in revenue. The transition to the GB300 has been seamless for major cloud service providers due to its shared architecture, software, and physical footprint. The GB200 enables them to build and deploy GB300 racks with ease. The transition to the new GB300 rack-based architecture has been seamless. Factory builds in late July and early August were successfully converted to support the GV300 ramp. Today, full production is underway. The current run rate is back at full speed, producing approximately 1,000 racks per week. This output is expected to accelerate even further throughout the third quarter as additional capacity comes online.

We expect widespread market availability in the second half of the year as CorWeave prepares to bring their GV300 instance to market as they are already seeing 10x more inference performance on reasoning models compared to H100. Compared to the previous Hopper generation, GV300 NDL72 AI factories promise a 10x improvement in token per watt energy efficiency, which translates to revenues as data centers are power limited.

The chips of the Rubin platform are in fab. The Vera CPU, Rubin GPU, CX9 Supernic, NBLink 144 scale-up switch, SpectrumX scale-out and scale-across switch, and the silicon photonics processor. Rubin remains on schedule for volume production next year. Rubin will be our third-generation NVLink Rack Scale AI supercomputer with a full-scale supply chain. This keeps us on track with our pace of an annual product cadence and continuous innovation across compute, networking, systems, and software.

In late July, the US government began reviewing licenses for sales of H20 to China customers. While a select number of our China-based customers have received licenses over the past few weeks, we have not shipped any H20 based on those licenses. USG officials have expressed an expectation that the US will receive 15% of the revenue generated from licensed H20 sales, but to date, the USG has not published a regulation codifying such a requirement. We have not included H20 in our Q3 outlook as we continue to work through geopolitical issues. If geopolitical issues resolve, we should ship $2 to $5 billion in H20 revenue in Q3. And if we had more orders, we could bill more.

We continue to advocate for the US government to approve Blackwell for China. Our products are designed and sold for beneficial commercial use, and every license sale we make will benefit the US economy, the US leadership in highly competitive markets. We want to win the hearts of every developer. America's AI technology stack can be the world's standard if we race and compete globally.

Notably in the quarter was an increase in Hopper 100 and H200 shipments. We also sold approximately $650 million of H20 in Q2 to an unrestricted customer outside of China. The sequential increase in Hopper demand indicates the breadth of data center workloads that run on acceler-

ated computing and the power of CUDA libraries and full stack optimizations, which continuously enhance the performance and economic value of our platform.

As we continue to deliver both Hopper and Blackwell GPUs, we are focusing on meeting the soaring global demand. This growth is fueled by capital expenditures from the cloud to enterprises, which are on track to invest $600 billion in data center infrastructure and compute this calendar year alone, nearly doubling in two years. We expect annual AI infrastructure investments to continue growing, driven by several factors: reasoning agentic AI requiring orders of magnitude more training and inference compute, global build-outs for Sovereign AI, enterprise AI adoption, and the arrival of physical AI and robotics.

Blackwell has set the benchmark as it is the new standard for AI inference performance. The market for AI inference is expanding rapidly with reasoning and agentic AI gaining traction across industries. Blackwell's RackScale NVLink and CUDA full stack architecture addresses this by redefining the economics of inference. New NVFP4 4-bit precision and NVLink 72 on the GB300 platform deliver a 50x increase in energy efficiency per token compared to Hopper, enabling companies to monetize their compute at unprecedented scale. For instance, a $3 million investment in GV200 can generate $30 million in token revenue, a 10x return.

NVIDIA Corporation's software innovation combined with the strength of our developer ecosystem has already improved Blackwell's performance by more than 2x since its launch. Advances in CUDA, TensorRT LLM, and Dynamo are unlocking maximum efficiency. CUDA library contributions from the open-source community along with NVIDIA Corporation's open libraries and frameworks are now integrated into millions of workflows. This powerful flywheel of collaborative innovation between NVIDIA Corporation and global community contribution strengthens NVIDIA Corporation's performance leadership.

NVIDIA Corporation is a top contributor to OpenAI models, data, and software. Blackwell has introduced a groundbreaking numerical approach to large language model pre-training. Using NBFP4 computations on the GB300 can now achieve 7x faster training than the H100, which uses FP8. This innovation delivers the accuracy of 16-bit precision with the speed and efficiency of 4-bit, setting a new standard for AI factor efficiency and scalability. The AI industry is quickly adopting this revolutionary technology, with major players such as AWS, Google Cloud, Microsoft Azure, and OpenAI, as well as Cohere, Mistral, Kimi AI, Perplexity, Reflection, and Runway, already embracing it.

NVIDIA Corporation's performance leadership was further validated in the latest MLPerf training benchmarks, where the GB200 delivered a clean sweep. Be on the lookout for the upcoming MLPerf inference results in September, which will include benchmarks based on the Blackwell Ultra.

NVIDIA RTX Pro servers are in full production for the world system makers. These are air-cooled PCIe-based systems integrated seamlessly into standard IT environments and run traditional enterprise workloads. Many global leaders are already adopting RTX Pro servers. Hitachi uses them for real-time simulation and digital twins, Lilly for drug discovery, Hyundai for factory design and AV validation, and Disney for immersive storytelling. As enterprises modernize data centers, RTX Pro servers are poised to become a multibillion-dollar product line.

Sovereign AI is on the rise as the nation's ability to develop its own AI using domestic infrastructure, data, and talent presents a significant opportunity for NVIDIA Corporation. NVIDIA Corporation is at the forefront of landmark initiatives across the UK and Europe. The European Union plans to

invest €20 billion to establish 20 AI factories across France, Germany, Italy, and Spain, including five gigafactories to increase its AI compute infrastructure tenfold. In the UK, the Isambard AI supercomputer powered by NVIDIA Corporation was unveiled as the country's most powerful AI system, delivering 21 exaflops of AI performance to accelerate breakthroughs in fields of drug discovery and climate modeling. We are on track to achieve over $20 billion in Sovereign AI revenue this year, more than double that of last year.

Networking delivered record revenue of $7.3 billion, and escalating demands of AI compute clusters necessitate high-performance and low-latency networking. This represents a 46% sequential and 98% year-on-year increase with strong demand across Spectrum X Ethernet, InfiniBand, and NVLink. Our Spectrum X enhanced Ethernet solutions provide the highest throughput and lowest latency network for Ethernet AI workloads. Spectrum X Ethernet delivered double-digit sequential and year-over-year growth with annualized revenue exceeding $10 billion.

At Hot Chips, we introduced Spectrum XGS Ethernet, a technology designed to unify disparate data centers into giga-scale AI super factories. CoreWeave is an initial adopter of the solution, which is projected to double GPU-to-GPU communication speed. InfiniBand revenue nearly doubled sequentially, fueled by the adoption of XDR technology, which provides double the bandwidth improvement over its predecessor, especially valuable for the model builders. The world's fastest switch, NVLink, with 14x the bandwidth of PCIe Gen 5, delivered strong growth as customers deployed large Blackwell and NVLink Rack Scale systems.

The positive reception to NVLink Fusion, which allows semi-custom AI infrastructure, has been widespread. Japan's upcoming Fugaku NEXT will integrate Fujitsu's CPUs with our architecture via NVLink Fusion. It will run a range of workloads, including AI, supercomputing, and quantum computing. Fugaku NEXT joins a rapidly expanding list of leading quantum supercomputing and research centers running on NVIDIA Corporation's CUDA-Q quantum platform, including ULEC, AIST, NNF, and NERSC, supported by over 300 ecosystem partners, including AWS, Google Quantum AI, Quantinuum, QEra, and SciQuantum.

Jetson Thor, our new robotics computing platform, is now available. Thor delivers an order of magnitude greater AI performance and energy efficiency than NVIDIA AGX Orin. It runs the latest generative and reasoning AI models at the edge in real time, enabling state-of-the-art robotics. Adoption of NVIDIA Corporation's robotics full stack platform is growing at a rapid rate. Over 2 million developers and 1,000+ hardware, software, application, and sensor partners are taking our platform to market. Leading enterprises across industries have adopted Thor, including Agility Robotics, Amazon Robotics, Boston Dynamics, Caterpillar, Figure, Hexagon, Medtronic, and Meta.

Robotic applications require exponentially more compute on the device and in infrastructure, representing a significant long-term demand driver for our data center platform. NVIDIA Omniverse with Cosmos is our data center physical AI digital platform built for the development of robot and robotic systems. This quarter, we announced a major expansion of our partnership with Siemens to enable AI automatic factories. Leading European robotics companies, including Agile Robots, Neura Robotics, and Universal Robots, are building their latest innovations with the Omniverse platform.

Transitioning to a quick summary of our revenue by geography. China declined on a sequential basis to low single digits percentage of data center revenue. Note, our Q3 outlook does not include

H20 shipments to China customers. Singapore revenue represented 22% of second quarter's billed revenue as customers have centralized their invoicing in Singapore. Over 99% of data center compute revenue billed to Singapore was for US-based customers.

Our gaming revenue was a record $4.3 billion, a 14% sequential increase and a 49% jump year on year. This was driven by the ramp of Blackwell GeForce GPUs as strong sales continued as we increase supply availability. This quarter, we shipped GeForce RTX 5060 desktop GPU. It brings double the performance along with advanced ray tracing, neural rendering, and AI-powered DLSS 4 to millions of gamers worldwide. Blackwell is coming to GeForce NOW in September. This is GeForce NOW's most significant upgrade, offering RTX 5080-class performance, minimal latency, and 5K resolution at 120 frames per second. We are also doubling the GeForce NOW catalog to over 4,500 titles, the largest library of any cloud gaming service.

For AI enthusiasts, on-device AI performs the best on RTX GPUs. We partnered with OpenAI to optimize their open-source GPT models for high-quality, fast, and efficient inference on millions of RTX-enabled Windows devices. With the RTX platform stack, Windows developers can create AI applications designed to run on the world's largest AI PC user base.

Professional visualization revenue reached $601 million, a 32% year-on-year increase. Growth was driven by the adoption of high-end RTX workstation GPUs and AI-powered workloads like design, simulation, and prototyping. Key customers are leveraging our solutions to accelerate their operations. Activision Blizzard uses RTX workstations to enhance creative workflows, while robotics innovator Figure AI powers its humanoid robots with RTX embedded GPUs.

Automotive revenue, which includes only in-car compute revenue, was $586 million, up 69% year on year, primarily driven by self-driving solutions. We have begun shipments of NVIDIA Thor SoC, the successor to Orin. Thor's arrival coincides with the industry's accelerating shift to vision-language model architecture, generative AI, and higher levels of autonomy. Thor is the most successful robotics and AV computer we have ever created and will power. Our full stack Drive AV software platform is now opening up billions of new revenue opportunities for NVIDIA Corporation while improving vehicle safety and autonomy.

Now moving to the rest of our P&L. GAAP gross margin was 72.4%, and non-GAAP gross margin was 72.7%. These figures include a $180 million or 40 basis point benefit from releasing previously reserved H20 inventory. Excluding this benefit, non-GAAP gross margins would have been 72.3%, still exceeding our outlook. GAAP operating expenses rose 86% on a non-GAAP basis sequentially. This increase was driven by higher compute and infrastructure costs as well as higher compensation and benefit costs.

To support the ramp of Blackwell and Blackwell Ultra, inventory increased sequentially from $11 billion to $15 billion in Q2. While we prioritize funding our growth and strategic initiatives in Q2, we returned $10 billion to shareholders through share repurchases and cash dividends. Our board of directors recently approved a $60 billion share repurchase authorization to add to our remaining $14.7 billion of authorization at the end of Q2.

Okay. Let me turn to the outlook for the third quarter. Total revenue is expected to be $54 billion plus or minus 2%. This represents over $7 billion in sequential growth. Again, we do not assume any H20 shipments to China customers in our outlook. GAAP and non-GAAP gross margins are expected to be 73.3% and 73.5%, respectively, plus or minus 50 basis points. We continue to expect to exit the year with non-GAAP gross margins in the mid-seventies. GAAP and non-GAAP

operating expenses are expected to be approximately $5.9 billion and $4.2 billion, respectively. For the full year, we expect operating expenses to grow in the high-thirties range year over year, up from our prior expectations of the mid-thirties. We are accelerating investments in the business to address the magnitude of growth opportunities that lie ahead.

GAAP and non-GAAP other income and expenses are expected to be an income of approximately $500 million, excluding gains and losses from non-marketable and publicly held equity securities. GAAP and non-GAAP tax rates are expected to be 16.5%, plus or minus 1%, excluding any discrete items. Further financial data are included in the CFO commentary and other information available on our website.

In closing, let me highlight upcoming events for the financial community. We will be at the Goldman Sachs Technology Conference on September 8 in San Francisco. Our annual NDR will commence in October. GTC Data Center begins on October 27 with Jensen's keynote scheduled for the twenty-eighth. We look forward to seeing you at these events. Our earnings call to discuss the results of our 2026 is scheduled for November 19. We will now open the call for questions.

Operator: Thank you. At this time, I would like to remind everyone in order to ask a question, press star, then the number one on your telephone keypad. Thank you. Your first question comes from C.J. Muse with Cantor Fitzgerald. Your line is open.

C.J. Muse (Analyst): I guess with wafer-in to rack-out lead times of twelve months, you confirmed on the call today that Rubin is on track for ramp in the second half. And, obviously, many of these investments are multiyear projects contingent upon power, cooling, etcetera. I was hoping perhaps you could take a high-level view and speak to your vision for growth into 2026. And as part of that, if you could kind of comment between network and data center, that would be very helpful. Thank you.

Jensen Huang (CEO): Thanks, C.J. At the highest level of growth, drivers would be the evolution, the introduction, if you will, of reasoning agentic AI. In a word, chatbots used to be one shot. You give it a prompt, and it would generate the answer. Now the AI does research, it thinks and does a plan. And it might use tools. And so it is called long thinking. And the longer it thinks, oftentimes, it produces better answers. And the amount of computation necessary for one shot versus reasoning agentic AI models could be 100 times, 1,000 times, and potentially even more as the amount of research and basically reading and comprehension that it goes off to do.

So the amount of computation that has resulted from agentic AI has grown tremendously. And, of course, the effectiveness has also grown tremendously. Because of agentic AI, the amount of hallucination has dropped significantly. You can now use tools and perform tasks. Enterprises have been opened up. As a result of agentic AI and vision-language models, we now are seeing a breakthrough in physical AI in robotics and autonomous systems. So in the last year, AI has made tremendous progress. And agentic systems, reasoning systems, are completely revolutionary.

Now we built the Blackwell NVLink 72 system, a rack-scale computing system for this moment. We have been working on it for several years. This last year, we transitioned from NVLink 8, which is a node-scale computing architecture, to now NVLink 72 where each rack is a computer. That disaggregation of NVLink 72 into a rack-scale system was extremely hard to do. But the results are extraordinary. We are seeing orders-of-magnitude speedup and, therefore, energy efficiency and, therefore, cost-effectiveness of token generation because of NVLink 72.

And so over the next couple of years, you are going to see growth. You asked about longer term. Over the next five years, we are gonna scale into, with Blackwell, with Rubin, and follow-ons, effectively a $3 to $4 trillion AI infrastructure opportunity. The last couple of years, you have seen that capex has grown in just the top four CSPs, doubled, and grown to about $600 billion. So we are in the beginning of this build-out, and the AI technology advances have really enabled AI to be able to adopt and solve problems for many different industries.

Operator: Your next question comes from Vivek Arya with Bank of America Securities. Your line is open.

Vivek Arya (Analyst): Colette, just wanted to clarify the $2 billion to $5 billion in China. What needs to happen, and what is the sustainable pace of that China business as you get into Q4? And then Jensen, for you on the competitive landscape, several of your large customers already have or are planning many ASIC projects. I think one of your ASIC competitors, Broadcom, signaled that they could grow their AI business almost 55–60% next year. Any scenario in which you see the market moving more towards ASICs and away from NVIDIA Corporation GPUs? Just what are you hearing from your customers? How are they managing this split between their use of merchant silicon and ASICs? Thank you.

Colette Kress (CFO): Thanks for that. So let me first answer your question regarding what will it take for the H20s to be shipped. There is interest in our H20s. There is the initial set of licenses that we received. And then additionally, we do have supply that we are ready, and that is why we communicated that somewhere in the range of about $2 to $5 billion this quarter we could potentially ship. We are still waiting on several of the geopolitical issues going back and forth between the governments and the companies trying to determine their purchases and what they want to do. So it is still open at this time, and we are not exactly sure what that full amount will be this quarter. However, if more interest arrives, more licenses arrive, again, we can also still build additional H20 and ship more as well.

Jensen Huang (CEO): NVIDIA Corporation builds very different things than ASICs. But let us talk about ASICs first. A lot of projects are started. Many startup companies are created. Very few products go into production. And the reason for that is it is really hard. Accelerated computing is unlike general-purpose computing. You do not write software and just compile it into a processor. Accelerated computing is a full stack co-design problem. And AI factories in the last several years have become so much more complex because the scale of the problems has grown so significantly. It is really the ultimate, most extreme computer science problem the world has ever seen.

The stack is complicated. The models are changing incredibly fast. From generative based on autoregressive to generative based on diffusion to mixed models to multimodality, the number of different models that are coming out that are either derivatives of transformers or evolutions of transformers is just daunting.

One of the advantages that we have is that NVIDIA Corporation is available in every cloud. We are available from every computer company. We are available from the cloud to on-prem to edge to robotics on the same programming model. And so it is sensible that every framework in the world supports NVIDIA Corporation. When you are building a new model architecture, releasing it on NVIDIA Corporation is most sensible.

And so the diversity of our platform, both in the ability to evolve into any architecture, the fact that we are everywhere, and also that we accelerate the entire pipeline—everything from data processing to pretraining to post-training with reinforcement learning all the way out to inference—means that when you build a data center with NVIDIA Corporation's platform in it, the utility of it is best. The lifetime usefulness is much, much longer.

And then I would just say that in addition to all of that, it is just an extremely complex systems problem anymore. People talk about the chip itself, the one ASIC, the GPU that many people talk about. But in order to build Blackwell, the platform, and Rubin, the platform, we had to build CPUs, memory, extremely energy-efficient memory for large KV caching necessary for agentic AI, a GPU, a SuperNIC, a scale-up switch we call NVLink, to a scale-out switch, whether it is Quantum or Spectrum X Ethernet, to now scale-across switches so that we could prepare for these AI super factories with multiple gigawatts of computing all connected together. We call that Spectrum XGS.

And then lastly, not only do we have the most energy-efficient architecture, our performance per watt is the best of any computing platform. And in a world of power-limited data centers, performance per watt drives directly to revenues. The more you buy, the more you grow. Because our performance per dollar is so incredible, you also have extremely great margins. So the growth opportunity with NVIDIA Corporation's architecture and the gross margin opportunity with NVIDIA Corporation's architecture is absolutely the best.

Operator: Your next question comes from Ben Reitzes with Melius. Your line is open.

Ben Reitzes (Analyst): Hey, thanks a lot. Jensen, I wanted to ask you about your $3 to $4 trillion in data center infrastructure spend by the end of the decade. Previously, you talked about something in the $1 trillion range, which I believe was just for compute. By 2028, if you take past comments, $3 to $4 trillion would imply maybe $2 trillion plus in compute spend. And just wanted to know if that was right and that is what you are seeing by the end of the decade. And wondering what you think your share will be of that. Your share right now of total infrastructure, compute-wise, is very high. Wanted to see if there are any bottlenecks you are concerned about, like power, to get to the $3 to $4 trillion. Thanks a lot.

Jensen Huang (CEO): Yeah. Thanks. As you know, the capex of just the top four hyperscalers has doubled in two years. As the AI revolution went into full steam, the AI race is now on, the capex spend has doubled to $600 billion per year. There are five years between now and the end of the decade. And $600 billion only represents the top four hyperscalers. We still have the rest of the enterprise companies building on-prem. You have cloud service providers building around the world. The United States represents about 60% of the world's compute. Over time, you would think that artificial intelligence would reflect GDP scale and growth.

Our contribution to that is a large part of the AI infrastructure. Out of a gigawatt AI factory, which can go anywhere from $50 to $60 billion, we represent about 35%, plus or minus. And of course, what you get for that is not a GPU. People think we are famous for building the GPU and inventing the GPU. But as you know, over the last decade, we have really transitioned to become an AI infrastructure company. It takes six different types of chips just to build a Rubin AI supercomputer.

The limiters going forward will likely be power limitations or AI building limitations. And so we

need to squeeze as much out of that factory as possible. NVIDIA Corporation's performance per unit of energy used drives the revenue growth of that factory. It directly translates. If you have a 100-megawatt factory, performance per 100 megawatts drives your revenues. In our case, also, the performance per dollar spent is so high that your gross margins are also the best. And so $3 to $4 trillion is fairly sensible for the next five years.

Operator: Next question comes from Joe Moore of Morgan Stanley. Your line is open.

Joe Moore (Analyst): Great. Thank you. Congratulations on reopening the China opportunity. Can you talk about the long-term prospects there? You have talked about, I think, half of the AI software world being there. How much can NVIDIA Corporation grow in that business? And how important is it that you get the Blackwell architecture ultimately licensed there?

Jensen Huang (CEO): The China market, I have estimated to be above $50 billion of opportunity for us this year. If we were able to address it with competitive products, and if it is $50 billion this year, you would expect it to grow, say, 50% per year as the rest of the world's AI market is growing as well. It is the second-largest computing market in the world, and it is also the home of AI researchers. About 50% of the world's AI researchers are in China. The vast majority of the leading open-source models are created in China. And so it is fairly important, I think, for American technology companies to be able to address that market.

Open source is really important for enterprise. It is really important for SaaS companies who also would like to build proprietary systems. It has been really incredible for robotics around the world. And it is important that American companies are able to address it. This is going to be a very large market. We are talking to the administration about the importance of American companies being able to address the Chinese market. As you know, H20 has been approved for companies that are not on the entities list. Many licenses have been approved. And so I think the opportunity for us to bring Blackwell to the China market is a real possibility.

Operator: Your next question comes from the line of Aaron Rakers with Wells Fargo. Your line is open.

Aaron Rakers (Analyst): Yes. Thank you for the question. I want to go back to the Spectrum XGS announcement this week and, thinking about the Ethernet product now pushing over $10 billion of annualized revenue, what is the opportunity set that you see for Spectrum XGS? Do you think about this as kind of the data center interconnect layer? Any thoughts on the sizing of this opportunity within that Ethernet portfolio? Thank you.

Jensen Huang (CEO): We now offer three networking technologies. One is for scale-up, one is for scale-out, and one for scale-across. Scale-up is so that we could build the largest possible virtual GPU, the virtual compute node. NVLink is revolutionary. NVLink 72 is what made it possible for Blackwell to deliver such an extraordinary generational jump over Hopper's NVLink 8.

We then scale out with networking, where we have two options. We have InfiniBand, which is unquestionably the lowest-latency, lowest-jitter, best scale-out network. For supercomputing and for the leading model makers, InfiniBand is the unambiguous choice. If you were to benchmark an AI factory, those with InfiniBand are the best-performing.

For those who would like to use Ethernet because their whole data center is built with Ethernet, we have a new type of Ethernet called Spectrum X Ethernet. It is not off the shelf. It has a whole bunch of new technologies designed for low latency, low jitter, congestion control, and it has the ability to come much closer to InfiniBand than anything that is out there.

And then finally, we have Spectrum XGS, a gigascale system for connecting multiple data centers, multiple AI factories into a super factory. A gigantic system. Networking is obviously very important in AI factories. Choosing the right networking and improving throughput effectively makes networking free because the improvement in system efficiency is so valuable. Spectrum X is now quite a sizable business, and it is only about a year and a half old. So Spectrum X is a home run. All three of them are going to be fantastic.

Operator: Your next question comes from Stacy Rasgon with Bernstein Research. Your line is open.

Stacy Rasgon (Analyst): Hi, guys. Thanks for taking my question. I have a more tactical question for Colette. So on the guide, we are up over $7 billion. The vast bulk of that is going to be from the data center. How do I think about apportioning that $7 billion out across Blackwell versus Hopper versus networking? It looks like Blackwell was probably $27 billion in the quarter up from maybe $23 billion last quarter. Hopper is still $6 or $7 billion post the H20. Do you think the Hopper strength continues? How do I think about parsing that $7 billion out across all those three different components?

Colette Kress (CFO): Thanks, Stacy, for the question. Looking at our growth between Q2 and Q3, Blackwell is still going to be the lion's share of what we have in terms of data center. But keep in mind that helps both our compute side as well as our networking side because we are selling those significant systems that are incorporating the NVLink that Jensen just spoke about. We are still selling Hopper, H100, and H200s. But again, I still believe Blackwell will be the lion's share of what we are doing. So Blackwell should again be the driver of the growth.

Operator: Your next question comes from Jim Schneider of Goldman Sachs. Your line is open.

Jim Schneider (Analyst): Good afternoon. Thanks for taking my question. You have been very clear about the reasoning model opportunity that you see and you have also been relatively clear about the technical specs for Rubin, but maybe you could provide a little bit of context about how you view the Rubin product transition going forward. What incremental capability does that offer to customers? And would you say that Rubin is a bigger, smaller, or similar step up in terms of performance or capability relative to what we saw with Blackwell? Thank you.

Jensen Huang (CEO): Rubin. We are on an annual cycle. And the reason why we are on an annual cycle is because we can do so to accelerate the cost reduction and maximize the revenue generation for our customers. When we increase the performance per watt, the token generation per amount of energy used, we are effectively driving the revenues of our customers. The performance per watt of Blackwell will be, for reasoning systems, an order of magnitude higher than Hopper. And so for the same amount of energy, and everybody's data center is energy-limited by definition, any data center using Blackwell will be able to maximize its revenues.

To the extent that we have great ideas for every single generation, we can improve revenue generation, improve AI capability, and improve the margins of our customers by releasing new architec-

tures. And so we advise our partners, our customers, to pace themselves and to build these data centers on an annual rhythm. Rubin is going to have a whole bunch of new ideas.

Operator: Your final question comes from Timothy Arcuri with UBS. Your line is open.

Timothy Arcuri (Analyst): Thanks a lot. Jensen, I wanted to ask: you threw out a number. You said 50% CAGR for the AI market. So I am wondering how much visibility you have into next year. Is that kind of a reasonable bogey in terms of how much your data center revenue should grow next year? I would think you will grow at least in line with that CAGR. Are there any puts and takes to that? Thanks.

Jensen Huang (CEO): Well, I think the best way to look at it is we have reasonable forecasts from our large customers for next year—a very, very significant forecast. And we still have a lot of businesses that we are still winning, and a lot of startups that are still being created. Do not forget that the number of AI-native startups funded was $100 billion last year. This year, the year is not even over yet, and it is $180 billion funded. If you look at the top AI-native startups that are generating revenues, last year it was $2 billion. This year it is $20 billion. Next year being 10 times higher than this year is not inconceivable.

The open-source models are now opening up large enterprises, SaaS companies, industrial companies, and robotics companies to join the AI revolution. We are just seeing enormous interest in AI and demand for AI. Right now, the buzz is that everything is sold out. H100s are sold out. H200s are sold out. Large CSPs are coming out renting capacity from other CSPs. AI-native startups are scrambling to get capacity so that they can train their reasoning models. And so the demand is really, really high.

Let me conclude with this. Blackwell is the next-generation AI platform the world has been waiting for. It delivers an exceptional generational leap. NVIDIA Corporation's NVLink 72 rack-scale computing is revolutionary, arriving just in time as reasoning AI models drive order-of-magnitude increases in training and inference performance requirements. Blackwell Ultra is ramping at full speed, and the demand is extraordinary. Our next platform, Rubin, is already in fab. We have six new chips that represent the Rubin platform. They have all taped out at TSMC. Rubin will be our third-generation NVLink rack-scale AI supercomputer. We expect to have a much more mature and fully scaled-up supply chain.

Blackwell and Rubin AI factory platforms will be scaling into the $3 to $4 trillion global AI factory build-out through the end of the decade. Customers are building ever-greater scale AI factories, from thousands of Hopper GPUs in tens-of-megawatt data centers to now hundreds of thousands of Blackwells in 100-megawatt facilities, and soon, we will be building millions of Rubin GPU platforms powering multi-gigawatt, multi-site AI super factories.

With each generation, demand only grows. One-shot chatbots have evolved into reasoning agentic AI that research, plan, and use tools, driving orders-of-magnitude jumps in compute for both training and inference. Agentic AI is reaching maturity and has opened the enterprise market to build domain- and company-specific AI agents for enterprise workflows, products, and services. The age of physical AI has arrived, unlocking entirely new industries in robotics and industrial automation. Every industry and every industrial company will need to build two factories: one to build the machines and another to build their robotic AI.

This quarter, NVIDIA Corporation reached record revenues and an extraordinary milestone in our

journey. The opportunity ahead is immense. A new industrial revolution has started. The AI race is on. Thanks for joining us today, and I look forward to addressing you on our next earnings call. Thank you.

Operator: This concludes today's conference call. You may now disconnect.

Appendix B

Sample from the Neo4j populated knowledge graph

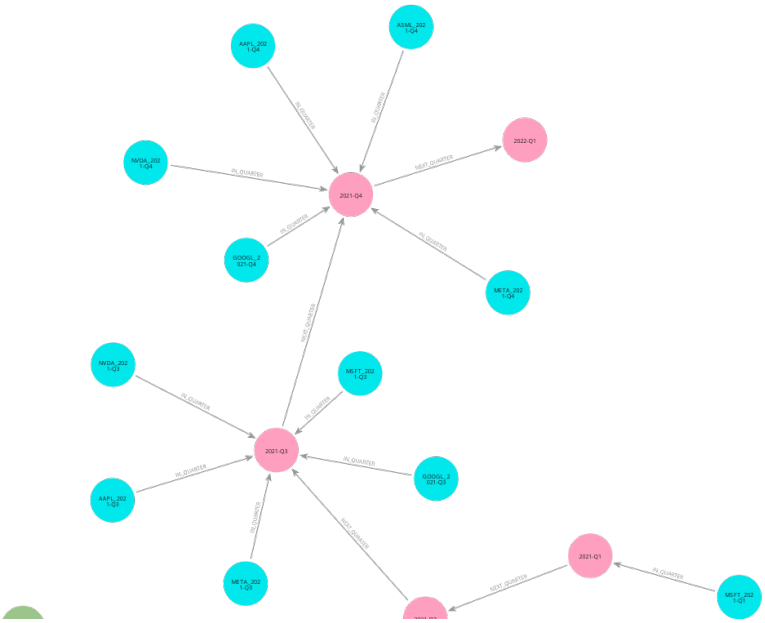


Figure B.1: Sample from the Knowledge Graph

Appendix C

Deployment Menu

This appendix summarizes how to use the IntelliTrade system for three main tasks: (1) train a model, (2) run inference or testing with a saved model, and (3) inspect existing analysis notebooks. All commands below assume that the repository root is the working directory.

C.1 Environment Setup

Before training or inference, the Python environment should be initialized and project dependencies installed:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

A root-level `.env` file is also required for external services. In practice, this includes Azure OpenAI credentials, Finnhub and Alpha Vantage API keys, and Neo4j credentials.

Configuring the `.env` File

Create a `.env` file in the project root with the following variables:

```
AZURE_OPENAI_API_KEY=
AZURE_OPENAI_ENDPOINT=https://your-resource-name.cognitiveservices.azure.com/
AZURE_API_VERSION=2024-12-01-preview
AZURE_DEPLOYMENT=gpt-5.2

FINNHUB_API_KEY=
ALPHA_VANTAGE_API_KEY=

NEO4J_URI=neo4j+s://xxxx.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=
```

The required credentials are:

`AZURE_OPENAI_API_KEY` Your Azure OpenAI API key.

`AZURE_OPENAI_ENDPOINT` The endpoint URL for your Azure OpenAI resource, typically of the form `https://your-resource-name.cognitiveservices.azure.com/`.

`AZURE_API_VERSION` The API version string, e.g. `2024-12-01-preview`.

`AZURE_DEPLOYMENT` The name of your deployed model, e.g. `gpt-5.2`.

`FINNHUB_API_KEY` Your Finnhub API key, used for market data retrieval.

`ALPHA_VANTAGE_API_KEY` Your Alpha Vantage API key, used for supplementary price data.

`NEO4J_URI` The connection URI for your Neo4j instance, e.g. `neo4j+s://xxxx.databases.neo4j.io`.

`NEO4J_USERNAME` The Neo4j database username (typically `neo4j`).

`NEO4J_PASSWORD` The Neo4j database password.

C.2 Train a Model

Model training is performed through the main CLI entry point `run.py`. The project supports three training methods:

- `dqn`: full context-enriched DQN using price, graph, and LLM-derived signals
- `dqn_nollm`: price-only DQN baseline
- `qlearning`: tabular Q-learning baseline

A representative command for training the full model is:

```
python run.py train --method dqn --epochs 50 --cutoff 2024-01-01
```

The corresponding price-only baseline is trained with:

```
python run.py train --method dqn_nollm --epochs 50 --cutoff 2024-01-01
```

Training Flags

The most important training flags are:

`-method` Selects the training algorithm. Valid options are `dqn`, `dqn_nollm`, and `qlearning`.

`-epochs` Sets the number of walk-forward training epochs. Larger values increase runtime but allow more repeated passes through the training universe.

`-cutoff` Defines the training cutoff date in `YYYY-MM-DD` format. Data before this date is used for training; later dates are typically reserved for inference and evaluation.

`-rebalance-frequency` Controls how often trading decisions are made. Valid options are `daily`, `weekly`, and `monthly`. Weekly is the default and is the main setting used in the project.

`-seed` Sets a random seed for reproducibility. This is especially useful for repeated benchmark runs.

Training output is written to `backtest_results/`. Typical final model artifacts include:

- `backtest_results/dqn_model_walkforward_final.pth`
- `backtest_results/dqn_nollm_model_walkforward_final.pth`

- `backtest_results/q_table_walkforward_final.json`

C.3 Run Inference (Test)

Inference is performed through the backtest interface with learning disabled. For the full DQN model, a typical frozen inference command is:

```
python run.py backtest --method dqn --ticker NVDA --no-learning \
  --load-model backtest_results/dqn_model_walkforward_final.pth \
  --start-date 2024-01-01
```

For the price-only baseline, the corresponding command is:

```
python run.py backtest --method dqn_nollm --ticker NVDA --no-learning \
  --load-model backtest_results/dqn_nollm_model_walkforward_final.pth \
  --start-date 2024-01-01
```

Inference and Backtest Flags

The most important backtest and inference flags are:

`-ticker` Specifies the stock ticker to evaluate, such as `NVDA`, `AAPL`, or `MSFT`.

`-method` Selects the policy type. Valid options are `fixed`, `qlearning`, `dqn`, and `dqn_nollm`. For frozen model inference, the main choices are `dqn` and `dqn_nollm`.

`-capital` Sets the initial portfolio value. The default is `10000.0`.

`-learning-rate` Sets the update step size. This mainly matters when learning is enabled.

`-epsilon` Sets the exploration rate. This has greater importance during learning than during frozen evaluation.

`-no-learning` Disables online learning during the backtest. This is the key option for inference because it ensures that the loaded model remains fixed.

`-no-save` Prevents the current run from writing a model checkpoint to disk.

`-no-confidence-sizing` Disables confidence-based position sizing and uses a simpler all-in or all-out allocation style.

`-stop-loss-pct` Applies a stop-loss threshold, written as a decimal. For example, `0.05` corresponds to a 5% stop-loss.

`-load-model` Loads a saved checkpoint from disk.

`-start-date` Sets the beginning of the evaluation period in `YYYY-MM-DD` format. In this project, inference commonly starts on `2024-01-01`.

`-rebalance-frequency` Sets the execution cadence for the run. Valid options are `daily`, `weekly`, and `monthly`.

Backtest results report performance statistics such as final portfolio value, total return, and Sharpe ratio. Saved histories and other evaluation artifacts are written to `test_results/` when applicable.

C.4 Compare Saved Models

For convenience, the project also provides a direct comparison command:

```
python run.py compare --ticker NVDA
```

This command automatically loads the default saved checkpoints for `dqn` and `dqn_nollm` and compares them on the same ticker.

The supported comparison flags are:

`-ticker` Selects the ticker to evaluate.

`-capital` Sets the initial capital.

`-stop-loss-pct` Adds a stop-loss threshold.

`-rebalance-frequency` Chooses the evaluation cadence.

The comparison command also writes ticker-level performance histories to `test_results/`.

C.5 View Existing Analysis

Existing analysis is provided through Jupyter notebooks in the `notebooks/` directory. A typical way to open them is:

```
jupyter lab notebooks/
```

The most relevant notebooks are:

- `notebooks/backtest_analysis.ipynb`, which provides multi-method comparisons
- `notebooks/backtest_analysis_daily.ipynb`, which analyzes daily-cadence experiments
- `notebooks/backtest_analysis_daily_dqn_nollm.ipynb`, which focuses on daily price-only experiments
- `notebooks/backtest_analysis_dqn_nollm.ipynb`, which summarizes no-LLM evaluation
- `notebooks/backtest_analysis_seeded_weekly_compare.ipynb`, which presents seeded weekly benchmark analysis

These notebooks are designed to work with project-relative directories such as `data/`, `backtest_results/`, and `test_results/`.